

Toegepaste Informatica

Bachelorproef

ondersteund door de

AP Hogeschool

begeleid door het bedrijf

Brightest

Milan Mareels

Begeleider: David Verhulst

Academiejaar 2025-2026 (2^{de} semester)

Voorwoord

Voor u ligt het definitieve sluitstuk van mijn opleiding Toegepaste Informatica, met de specialisatie IT & Software, aan de AP Hogeschool Antwerpen. Dit document vormt de bundeling van mijn bachelorproef en markeert de overgang van mijn studententijd naar een professionele carrière in de IT-sector. Het project dat hierin centraal staat, de BrightSwagShop, is het resultaat van een intensieve stageperiode bij Brightest, waarbij ik de kans kreeg om complexe software te bouwen, ondersteund door de diepgaande expertise van het bedrijf op het gebied van softwarekwaliteit.

Deze bachelorproef is opgebouwd uit vier essentiële pijlers die het volledige traject van een softwareproject weerspiegelen. Het traject startte bij de Blueprint, waarin de initiële analyse en strategische visie werden vastgelegd en de fundamentele behoefte van Brightest aan een educatief platform met bug toggles werd vertaald naar concrete doelstellingen. Omdat de BrightSwagShop een instrument is voor testconsultants, was het Testplan een onmisbaar onderdeel van mijn werk. Hierin wordt beschreven hoe we via Jest en Playwright niet alleen de stabiliteit van de fake webshop garanderen, maar ook de correcte werking van de opzettelijk geïnjecteerde defecten valideren. Vervolgens biedt het Test Reflectieverslag een kritische terugblik op deze testfase, hierin bespreek ik de concrete resultaten die de testen hebben opgeleverd en analyseer ik de obstakels waar ik tijdens het testen tegenaan ben gelopen. Tot slot vormt de Technische Documentatie het hart van de realisatie. Dit document beschrijft in detail de technische werking van het systeem en hoe de applicatie wordt gedeployd. Hierin worden de gekozen technologieën, zoals Next.js, Express, Prisma en Docker, toegelicht en wordt het concrete realisatieplan uiteengezet. Daarnaast biedt het document een compleet overzicht van de gelaagde architectuur, de focus op security en de aanpak voor de datamigratie.

Dankwoord

Graag wil ik een aantal personen bedanken die hebben bijgedragen aan de realisatie van dit project en de begeleiding van mijn stage.

Allereerst gaat mijn dank uit naar Cat Ooms, mijn Project Manager bij Brightest. Bedankt voor de nauwe opvolging van mijn stage en de kans om deel uit te maken van jullie team. Jouw begeleiding heeft ervoor gezorgd dat het project steeds op koers bleef.

Daarnaast bedank ik Sepp Van Cauwenbergh, onze Product Owner, voor het definiëren van de visie van de BrightSwagShop en het bewaken van de functionele vereisten van dit platform.

Ook wil ik Ben Van de Velden uitdrukkelijk bedanken. Jouw technische ondersteuning bij complexe problemen was van onschatbare waarde om de technische uitdagingen van dit project te overwinnen.

Verder bedank ik David Verhulst, mijn stagebegeleider vanuit de AP Hogeschool, voor de sturing en de feedback vanuit de opleiding gedurende de verschillende fases van dit project.

Tot slot bedank ik de AP Hogeschool voor de geboden kennis en ondersteuning gedurende mijn volledige opleiding.

Inhoudstafel

VOORWOORD	2
DANKWOORD	2
TERMEN EN AFKORTINGEN	5
INLEIDING.....	6
BLUEPRINT	7
I. BEDRIJFSCONTEXT	8
A. OPDRACHTGEVER.....	8
B. HUIDIGE SITUATIE	8
C. TOEKOMSTIGE SITUATIE	8
II. OPLOSSING	14
A. DOELSTELLING.....	14
B. FUNCTIONELE SCOPE	14
1. Use case 1: Nieuwe instantie aanmaken	17
2. Use case 2: Bugs Configureren	17
3. Use case 3: Bestelling Plaatsen.....	17
4. Use case 4: Inloggen	18
5. Use case 5: Testdata Resetten	18
6. Use case 6: Product Toevoegen	19
7. Use case 7: Instantie Selecteren	19
8. Use case 8: Winkelmandje Beheren.....	20
9. Use case 9: Bestelgeschiedenis Bekijken.....	20
10. Use case 10: Gebruikers aanmaken	21
C. FUNCTIONEEL DESIGN	21
1. Scherm 1: Inloggen	25
2. Scherm 2: Home.....	26
3. Scherm 3: Product details	27
4. Scherm 4: Winkelmandje	28
5. Scherm 5: Bestelling afronden	29
6. Scherm 6: Contact.....	30
7. Scherm 7: Admin dashboard.....	31
8. Scherm 8: Super Admin dashboard.....	32
9. Scherm 9: Bug configuratie.....	33
10. Scherm 10: Admin aanmaken	34
11. Scherm 11: Admin Beheer	35
12. Scherm 12: Data reset	36
13. Scherm 13: Instantie overzicht Super Admin.....	37
14. Scherm 14: User overzicht per instantie Admin.....	38
15. Scherm 15: User overzicht per instantie Super Admin.....	39
16. Scherm 16: Gebruiker aanmaken pop-up.....	40
17. Scherm 17: Instantie aanmaken pop-up	41
18. Scherm 18: Geen toegang pop-up	42
19. Scherm 19: Instantie details	43
20. Scherm 20: Register.....	44
21. Scherm 21: Bestelling geschiedenis	45
D. INFORMATIE.....	46
E. TECHNISCH DESIGN.....	47
F. DOCUMENTATIE	55
III. PLANNING	56
IV. SAMENVATTING	59
TESTPLAN	60

A.	INLEIDING	61
B.	PROJECTBESCHRIJVING EN SCOPE.....	61
C.	BELANGHEBBENDEN	61
D.	RISICOANALYSE.....	62
1.	<i>Data overslag tussen gescheiden instanties.....</i>	63
2.	<i>Bug configuraties lekken naar ongeautoriseerde omgevingen</i>	63
3.	<i>SQL-Injectie door testscripts</i>	63
4.	<i>API overbelasting door scripts cursisten</i>	64
5.	<i>Frontend caching toont oude bug-status</i>	64
6.	<i>Ongeautoriseerde toegang tot beheerpaneel</i>	64
7.	<i>Bug-toggle genegeerd door backend logica.....</i>	64
8.	<i>Data Reset botst met actieve testscripts</i>	65
9.	<i>Swagger API documentatie loopt asynchroon.....</i>	65
10.	<i>Databank connectie-pool uitgeput.....</i>	65
E.	TESTSTRATEGIE	67
F.	TESTOMGEVING.....	68
G.	TESTRESULTATEN	69
	TEST REFLECTIEVERSLAG	70
H.	INLEIDING	71
I.	REFLECTIE	71
J.	CONCLUSIE	73
K.	TESTRESULTATEN	73
	TECHNISCHE DOCUMENTATIE	74
L.	SAMENVATTING VAN DE OPDRACHT	75
M.	IMPACT OP DE INFRASTRUCTUUR	75
N.	RELEASE PLAN	76
O.	TECHNISCH DESIGN	78
P.	EXTERNE SYSTEEMINTERFACES	80
Q.	DATAMIGRATIE	81
R.	SECURITY EN AUTORISATIEROLLEN	81
S.	DOCUMENTATIE	82
	SAMENVATTING	83
	BESLUIT	83
	BRONVERMELDING.....	84
	BIJLAGEN	85

Termen en Afkortingen

Term	Omschrijving
API	Application Programming Interface
CD	Continuous Deployment
CI	Continuous Integration
CORS	Cross Origin Resource Sharing
E2E	End-To-End
ENV	Environment Variables
HTTP	Hyper Text Transfer Protocol
HTTPS	Hyper Text Transfer Protocol Secure
JSON	Javascript Object Notation
JWT	JSON Web Token
ORM	Object Relational Mapping
PR	Pull Request
SEO	Search Engine Optimization
UI	User Interface
UX	User Experience

Inleiding

In opdracht van Brightest is de BrightSwagShop ontwikkeld: een fake e-commerce omgeving die specifiek dient als trainingsplatform voor testconsultants. Waar de meeste webshops streven naar perfectie, is dit platform uniek omdat fouten doelbewust en gecontroleerd geactiveerd kunnen worden via een dashboard. Dit stelt consultants in staat om hun testvaardigheden te trainen in een realistische, maar veilige omgeving.

Dit dossier biedt een allesomvattend overzicht van het project en is opgebouwd uit vier cruciale onderdelen. Allereerst is er de Blueprint, een strategische analyse waarin de functionele vereisten en de noodzaak van het project worden vastgelegd. Daarnaast bevat het document het Testplan, dat een gedetailleerde beschrijving geeft van hoe de kwaliteit van zowel de shop als de bug toggles wordt gewaarborgd via geautomatiseerde testen. Vervolgens volgt het Test Reflectieverslag, met een kritische analyse van de testresultaten en de technische uitdagingen die tijdens de ontwikkelingsfase naar voren kwamen. Tot slot biedt de Technische Documentatie een diepgaande blik op de architectuur, de multi-tenancy infrastructuur en de gebruikte technologieën zoals Next.js, Express en Prisma. Hiermee wordt aangetoond hoe een moderne full-stack applicatie kan worden ingezet voor educatieve doeleinden binnen de QA-sector.

Toegepaste Informatica

Blueprint BrightSwagShop

ondersteund door de

AP Hogeschool

begeleid door het bedrijf

Brightest

Milan Mareels

Begeleider: David Verhulst

Academiejaar 2025-2026 (2^{de} semester)

I. Bedrijfscontext

A. Opdrachtgever

Brightest is opgericht in 2016 en is gespecialiseerd in testautomatisering en het optimaliseren van IT-processen. Met meer dan 130 consultants en kantoren in België en Nederland ondersteunt de organisatie bedrijven bij het realiseren van hun kwaliteitsdoelstellingen.

De opdrachtgever voor deze stageopdracht is Sepp Van Cauwenbergh. Sepp vervult de rol van product owner binnen Brightest en is ook de initiatiefnemer van dit project. Een van zijn belangrijkste drijfveren voor deze opdracht is het faciliteren van trainingen, demo's en recruiting door middel van een nieuw testobject. Hij streeft naar een omgeving waarin de kracht van testen en testautomatisering beter gedemonstreerd kunnen worden, specifiek door de mogelijkheid te bieden om bugs op commando aan en uit te zetten.

B. Huidige Situatie

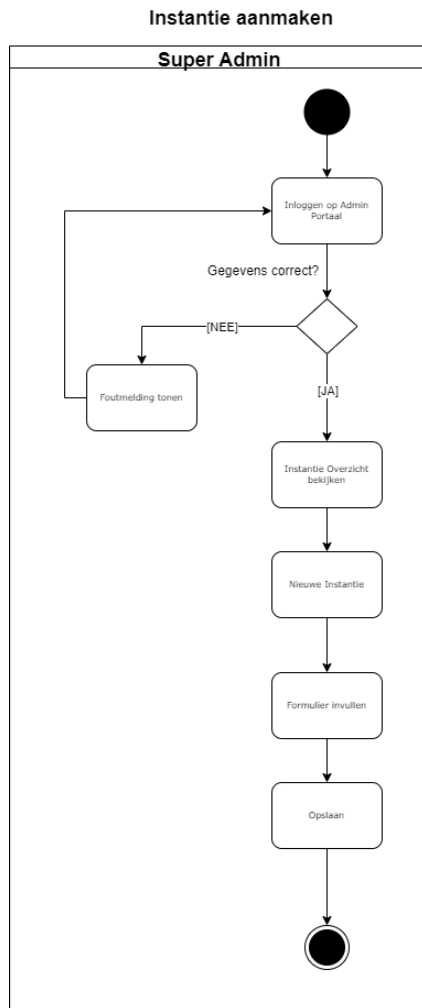
Momenteel beschikt de organisatie echter niet over een specifieke applicatie die effectief kan dienen als een testobject voor doeleinden zoals trainingen, demo's en recruiting. Wanneer Brightest nu de kracht van test-automation wil demonstreren aan klanten, of interne trainingen wil geven, ontbreekt er een gecontroleerde omgeving waarin "bugs" in de software gesimuleerd kunnen worden. Er is geen "tool" beschikbaar waarin bugs op commando aan- en uitgezet kunnen worden.

Doordat er geen specifieke applicatie aanwezig is, is het lastig om aan te tonen hoe testautomatisering reageert op functionaliteiten die zich niet goed gedragen, dit is essentieel voor een overtuigende demo. Ook voor interne trainingen vormt de afwezigheid van dit platform een beperking. Zonder een omgeving met gecontroleerde bugs hebben trainers geen goede basis om nieuwe testers of studenten gericht te trainen in het opsporen van fouten.

C. Toekomstige Situatie

Om de huidige uitdagingen rondom demo's, trainingen en recruiting aan te pakken, is een gecontroleerde en flexibele testomgeving noodzakelijk. In de toekomstige situatie wordt de BrightSwagShop gerealiseerd, een applicatie die specifiek is ontwikkeld om te dienen als testobject voor trainingen, demo's en recruiting.

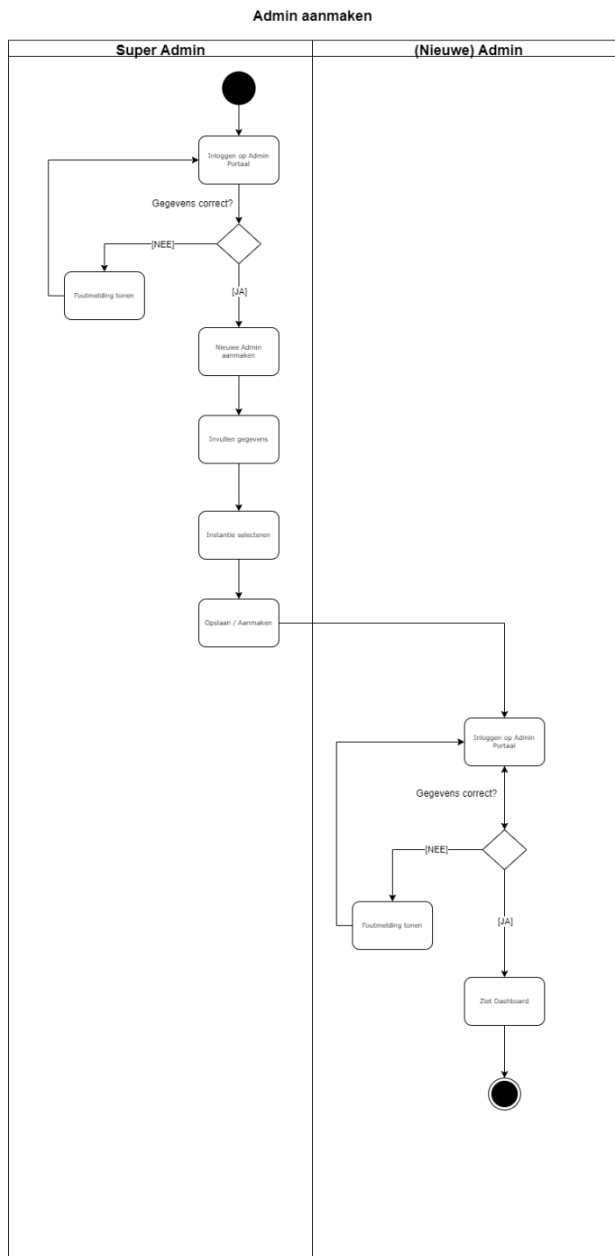
Deze applicatie fungeert als een fake webshop waar gebruikers Brightest promotiemateriaal kunnen aankopen, met als unieke eigenschap dat er opzettelijk bugs zijn ingebouwd.



Figuur 1. Toekomstige situatie instantie aanmaken

Dit activiteitendiagram visualiseert de beheertaken van de Super Admin, een rol die doorgaans wordt vervuld door een medewerker van Brightest. Het proces begint wanneer de Super Admin inlogt op het systeem om de infrastructuur voor een nieuwe klant, training of recruiting voor te bereiden. In plaats van dat de klant zelf een omgeving moet installeren, faciliteert de Super Admin dit centraal door via het dashboard een nieuwe unieke instantie aan te maken.

Tijdens dit proces voert de Super Admin de specifieke klantgegevens in. Zodra deze gegevens zijn opgeslagen, genereert het systeem volledig automatisch een afgeschermd omgeving. Deze handeling vormt de fundering om te garanderen dat de configuraties van de nieuwe klant strikt gescheiden blijven van andere gebruikers.



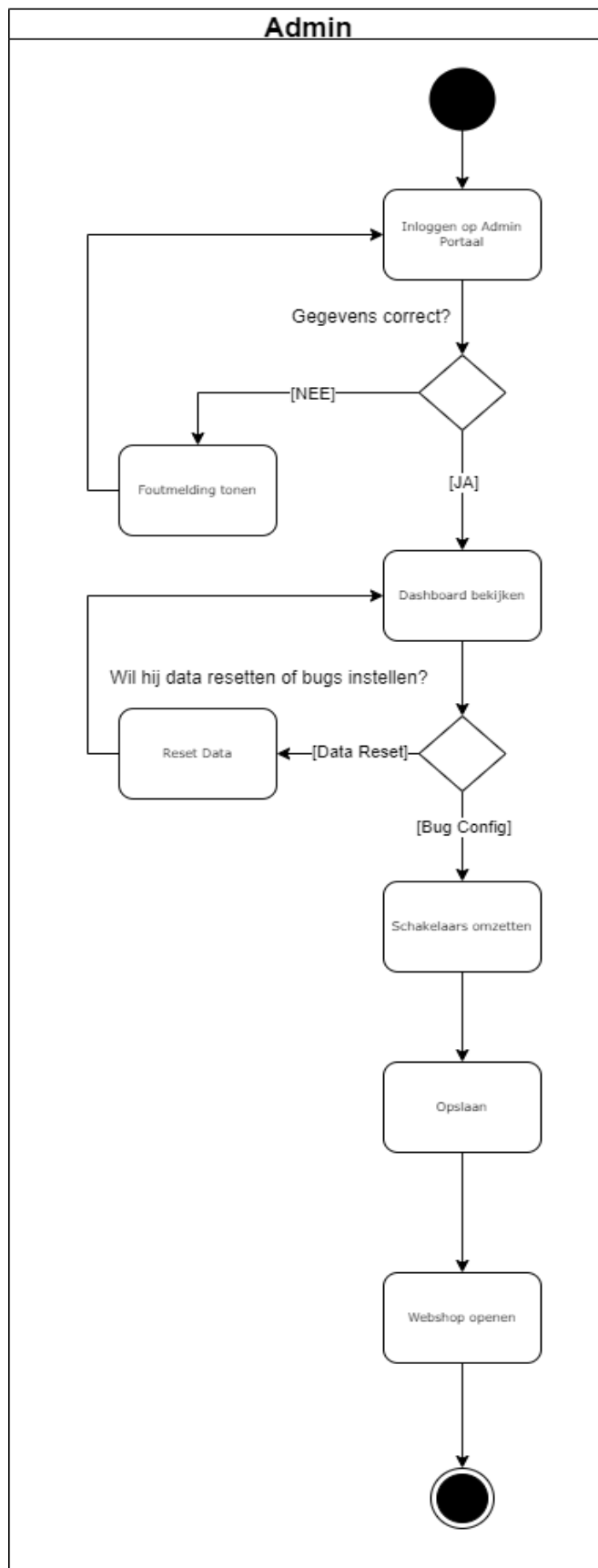
Figuur 2. Toekomstige situatie admin aanmaken

Dit activiteitendiagram toont de vervolgstap op het creëren van een nieuwe instantie, waarin de Super Admin een beheerdersaccount aanmaakt voor de klant of trainer. Nadat de technische instantie is gerealiseerd, kan de Super Admin een nieuwe Gebruiker of Admin aan maken. Hierbij worden niet alleen de persoonsgegevens ingevoerd, maar wordt de nieuwe Gebruiker of Admin ook expliciet gekoppeld aan de juist aangemaakte instantie.

Deze koppeling is cruciaal binnen het systeem, omdat het garandeert dat de nieuwe Gebruiker of Admin enkel toegang heeft tot zijn eigen data en configuraties, zonder inzicht in andere omgevingen.

Zodra het account is opgeslagen, kan de nieuwe Gebruiker of Admin inloggen op het systeem.

Training Voorbereiden

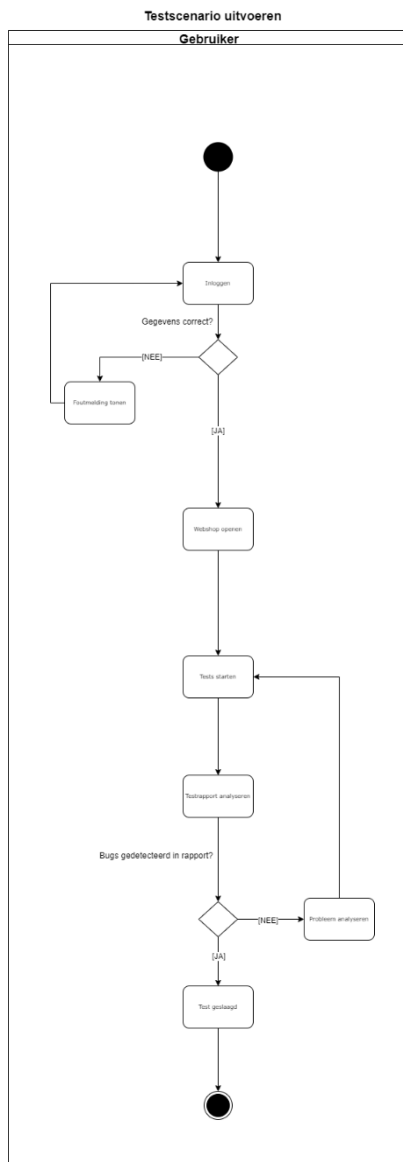


Figuur 3. Toekomstige situatie training voorbereiden

Dit activiteitendiagram visualiseert de workflow van de Admin bij het inrichten van de omgeving voor een specifieke testsessie. Nadat de Admin heeft ingelogd op het systeem, komt hij terecht op het dashboard waar hij de status van zijn instantie kan beheren. Vanuit hier faciliteert het systeem twee essentiële beheeracties om een training, demo of recruiting voor te bereiden.

Ten eerste biedt het systeem de mogelijkheid aan om testdata te resetten. Dit is een cruciale stap bij de start van een nieuwe groep, demo of recruiting. Hiermee kan de Admin in één keer alle vervuilde data zoals orders die door vorige gebruikers zijn aangemaakt verwijderen om met een schone lei te beginnen.

Ten tweede kan de Admin via de bug configuratie bepalen welk scenario er gespeeld wordt. In plaats van te zoeken naar toevallige fouten, zet de Admin hier doelbewust specifieke bugs aan of uit door middel van schakelaars. Nadat de gewenste configuratie opgeslagen is, opent de Admin de webshop om te verifiëren dat de omgeving zich gedraagt zoals verwacht.



Figuur 4. Toekomstige situatie testscenario uitvoeren

Dit activiteitendiagram visualiseert de flow voor de eindgebruiker. Hierbij wordt ervan uitgegaan dat de gebruiker reeds beschikt over een geregistreerd account dat gekoppeld is aan een specifieke instantie.

Het proces is hierdoor gestroomlijnd, de gebruiker logt in met de correcte gegevens en komt direct terecht in zijn eigen, afgeschermd testomgeving. Zodra de webshop is geopend, start de kernactiviteit, het uitvoeren van de manuele of geautomatiseerde testscripts. De gebruiker analyseert vervolgens het testrapport om te valideren of de scripts correct hebben gefunctioneerd. Indien de geconfigureerde bugs niet zijn gedetecteerd, volgt een cyclus van debuggen en controleren. Pas wanneer de bugs succesvol in het rapport verschijnen, is het leerdoel bereikt en de test geslaagd.

MVP Scope en Prioritering

Gezien de tijdgebonden aard van dit project ligt de volledige focus op de kern van de applicatie. De absolute prioriteit is het realiseren van een robuust, eigen authenticatiesysteem. Pas wanneer deze en alle overige kernfunctionaliteiten volledig operationeel en stabiel zijn, ontstaat er ruimte voor verdere technische uitbreidingen.

Op dit moment wordt de applicatie uitsluitend ontwikkeld om lokaal te draaien. Toch zal er in de huidige opzet al rekening gehouden worden met een toekomstige transitie naar een live-omgeving. De architectuur wordt zo ingericht dat de stap naar online hosting en schaalbaarheid in een later stadium soepel kan worden uitgevoerd.

In deze toekomstige situatie zijn diverse optimalisaties aangemerkt als Nice To Have functionaliteiten. Een essentieel onderdeel hiervan is de inrichting van CI/CD pipelines voor test automation met frameworks zoals Jest en Playwright. Hoewel deze tools de softwarekwaliteit en het deployment-proces aanzienlijk versterken, worden ze pas geïmplementeerd nadat de primaire MVP-doelstellingen succesvol zijn behaald en de overgang naar een online omgeving aan de orde is.

II. Oplossing

A. Doelstelling

Het doel is het ontwikkelen van een fake webshop (BrightSwagShop), een applicatie die dient als testobject voor trainingen, demo's en recruiting. Door dit platform worden de volgende problemen opgelost en opportuniteiten benut:

a) Opgeloste problemen

- **Ineffectieve demonstraties door gebrek aan 'foute' software:** momenteel moeten demo's vaak gegeven worden op applicaties die te stabiel zijn of zich onvoorspelbaar gedragen. Hierdoor is het bedrijf niet in staat om aan klanten te tonen hoe testautomatisering reageert op specifieke fouten. De nieuwe applicatie lost dit op door bugs aan te bieden die gegarandeerd optreden. Dit zorgt ervoor dat we een gecontroleerde omgeving hebben om opzettelijke bugs aan en uit te zetten.
- **Gebrek aan rekruterings testen:** er is geen uniform platform beschikbaar om de technische skills van sollicitanten objectief te meten. Door de BrightSwagShop te gebruiken als vaste case, kan elke sollicitant op exact dezelfde bugs en scenario's worden getest.
- **Tijdsverlies bij voorbereiding van trainingen:** trainers verliezen veel tijd met het zoeken naar geschikte oefenmaterialen. Door de BrightSwagShop aan te bieden die zowel web, mobile als backend bevat, wordt de voorbereidingstijd geminimaliseerd.

b) Benutte Opportuniteiten

- **Controle via bug toggles:** we creëren de mogelijkheid om bugs via een adminpaneel aan en uit te zetten. Dit maakt de applicatie multifunctioneel: in de basisstaat dient het als een fake webshop, en in de "buggy" staat dient het als testobject.
- **Schaalbaarheid door gescheiden instanties:** de applicatie is in staat om via een Super Admin functionaliteiten volledig gescheiden aan te bieden. Hierdoor kunnen meerdere trainingen of demo's gelijktijdig plaatsvinden in hun eigen omgeving, zonder dat data of configuraties elkaar overschrijven.
- **Volledige technische dekking:** de applicatie beperkt zich niet enkel tot frontend, het biedt ook een volledige REST backend. Dit biedt de kans om trainingen of demo's uit te breiden naar API-testen en integratietesten.

B. Functionele Scope

In de functionele scope van dit project geven we een gedetailleerd overzicht van de belangrijkste functionaliteiten die worden geïmplementeerd. Het bevat een Use Case Diagram met bijhorende scenario's. Daarnaast worden de schermen weergegeven, aangevuld met toelichtingen, om een duidelijk beeld te schetsen van de gebruikersinterface en de werking van het systeem.

Niet in de functionele scope:

- **E-mail notificaties:** het systeem verstuurt geen daadwerkelijke bevestigingsmails bij het aanmaken van een account of het plaatsen van een bestelling.

- **Echte betalingsverwerking:** er wordt geen integratie gebouwd met echte payment providers zoals Stripe, Klarna of Paypal. Alle betalingen worden intern door de backend gesimuleerd.
- **Voorraadbeheer:** het systeem houdt geen rekening met de daadwerkelijke voorraad van producten. Een product kan oneindig besteld worden en zal nooit uitverkocht zijn.
- **Track & Trace integratie:** het verzenden en volgen van pakketten via externe koeriersdiensten valt buiten het project.
- **Wachtwoord resetten:** er wordt geen functionaliteit gebouwd om vergeten wachtwoorden te herstellen. Gebruikers kunnen hun wachtwoord dus niet zelf aanpassen als ze dit kwijt zijn.



1. Use case 1: Nieuwe instantie aanmaken

Actor	Super Admin
Aannamen	De actor is succesvol ingelogd op het beheerportaal en bevindt zich op het instantie overzicht scherm.
Beschrijving	1. De actor klikt op de knop om een nieuwe instantie aan te maken. 2. Het systeem toont het aanmaakformulier. 3. De actor vult de juiste gegevens in. 4. De actor klikt op de knop om de instantie aan te maken. 5. Het systeem valideert de invoer. 6. Het systeem maakt de nieuwe instantie en genereert de standaard bug configuratie. 7. Het systeem toont de nieuwe instantie in het overzicht.
Uitzonderingen	5A [Bestaande instantie naam]
Resultaat	Er is een nieuwe instantie met bijhorende bug configuratie opgeslagen in het systeem.

Alternatieve scenario's

Naam	5A [Bestaande instantie naam]
Beschrijving	1. Het systeem toont een foutboodschap. 2. De actor voert een andere naam in. 3. Terug naar basis scenario vanaf stap 4.

2. Use case 2: Bugs Configureren

Actor	Super Admin, Admin
Aannamen	De actor is succesvol ingelogd op het beheerportaal.
Beschrijving	1. De actor navigeert naar het menu om de bugs te configureren. 2. Het systeem haalt alle bugs op. 3. De actor zet een bug toggle aan. 4. De actor klikt op de knop om op te slaan. 5. Het systeem werkt de database bij. 6. Het systeem toont een succesmelding.
Uitzonderingen	5A [Opslaan mislukt]
Resultaat	De actor heeft bugs aan en uit kunnen zetten, en de webshop toont het gewenste foutieve gedrag.

Alternatieve scenario's

Naam	5A [Opslaan mislukt]
Beschrijving	1. Het systeem toont een foutboodschap. 2. De toggle springt naar de originele stand. 3. De actor probeert het opnieuw. 4. Terug naar basis scenario stap 4.

3. Use case 3: Bestelling Plaatsen

Actor	Gebruiker
-------	-----------

Aannamen	De actor is succesvol ingelogd, heeft een instantie gekozen en producten in het winkelmandje.
Beschrijving	1. De actor opent het winkelmandje en klikt op de knop om af te rekenen. 2. Het systeem toont het besteloverzicht. 3. De actor vult de juiste adresgegevens in. 4. De actor klikt op de knop om te betalen. 5. Het systeem verwerkt de fictieve betaling. 6. Het systeem maakt de order in de database. 7. Het systeem leegt het winkelmandje. 8. Het systeem toont de bedankpagina.
Uitzonderingen	4A [Betaling mislukt]
Resultaat	De actor heeft succesvol een bestelling kunnen plaatsen.

Alternatieve scenario's

Naam	4A [Betaling mislukt]
Beschrijving	1. Het systeem toont een foutboodschap. 2. De bestelling wordt niet afgerond. 3. De actor kiest een andere betaalmethode of probeert opnieuw 4. Terug naar basis scenario stap 4.

4. Use case 4: Inloggen

Actor	Super Admin, Admin, Gebruiker
Aannamen	De actor bevindt zich op de inlogpagina en heeft een account.
Beschrijving	1. De actor voert zijn e-mailadres en wachtwoord in. 2. De actor klikt op de knop om in te loggen. 3. Het systeem verifieert de inloggegevens. 4. Het systeem leidt de gebruiker door naar het dashboard dat bij zijn rol behoort.
Uitzonderingen	3A [Foutieve inloggegevens]
Resultaat	De gebruiker is geauthentiseerd en heeft toegang tot het platform.

Alternatieve scenario's

Naam	3A [Foutieve inloggegevens]
Beschrijving	1. Het systeem toont een foutboodschap. 2. Het systeem vraagt om opnieuw de correcte gegevens in te vullen. 3. Terug naar basis scenario stap 2.

5. Use case 5: Testdata Resetten

Actor	Super Admin, Admin
Aannamen	De actor is ingelogd en er zitten bestellingen in de database.
Beschrijving	1. De actor navigeert naar het test data reset scherm. 2. De actor klikt op de knop om de data te resetten. 3. Het systeem toont een waarschuwingpop-up

	- De actor bevestigt zijn actie. - Het systeem verwijdert alle orders van deze omgeving. - Het systeem toont een succesmelding.
Uitzonderingen	4A [Annuleren]
Resultaat	De actor heeft de bestelhistorie van deze instantie permanent kunnen verwijderen.

Alternatieve scenario's

Naam	4A [Annuleren]
Beschrijving	- De actor ziet de waarschuwingpop-up. - De actor klikt op de knop om te annuleren. - Het systeem sluit de pop-up en er wordt geen data verwijderd. - Terug naar basis scenario stap 1.

6. Use case 6: Product Toevoegen

Actor	Super Admin
Aannamen	De actor is succesvol ingelogd op het beheerportaal en bevindt zich op het producten overzicht scherm.
Beschrijving	- De actor klikt op de knop om een nieuw product aan te maken. - De actor vult het invulformulier in. - De actor klikt op de knop om op te slaan. - Het systeem valideert de gegevens. - Het systeem slaat het product op.
Uitzonderingen	4A [Ongeldige invoergegevens]
Resultaat	De actor heeft een nieuw product kunnen toevoegen aan de catalogus.

Alternatieve scenario's

Naam	4A [Ongeldige invoergegevens]
Beschrijving	- Het systeem toont een foutboodschap. - Het systeem vraagt om het formulier terug bij te werken. - De actor vult de juiste gegevens in. - Terug naar basis scenario stap 3.

7. Use case 7: Instantie Selecteren

Actor	Super Admin, Admin
Aannamen	De actor is net ingelogd maar heeft nog geen omgeving gekozen.
Beschrijving	- Het systeem toont een lijst met beschikbare omgevingen. - De actor klikt op de gewenste omgeving. - De actor bevestigt. - Het systeem slaat de wijziging op. - Het systeem stuurt de actor naar de homepage.
Uitzonderingen	1A [Geen beschikbare instanties]
Resultaat	De actor bevindt zich in de juiste, geïsoleerde webshop-omgeving.

Alternatieve scenario's

Naam	1A [Geen beschikbare instanties]
Beschrijving	1. Het systeem toont een foutboodschap. 2. De actor contacteert de helpdesk. 3. De actor wacht tot het probleem opgelost is. 4. Terug naar basis scenario stap 1.

8. Use case 8: Winkelmandje Beheren

Actor	Gebruiker
Aannamen	De actor is net ingelogd en bevindt zich op de homepage.
Beschrijving	1. De actor selecteert het item dat die wil toevoegen aan zijn winkelwagen. 2. De actor klikt op de knop om het item toe te voegen aan zijn winkelwagen. 3. Het systeem voegt het item toe aan de lijst. 4. Het systeem toont een notificatie dat het product toegevoegd is. 5. De items van het winkelmandje worden bijgewerkt.
Uitzonderingen	2A [Item kan niet worden toegevoegd]
Resultaat	De actor heeft een item toegevoegd aan zijn winkelmandje.

Alternatieve scenario's

Naam	2A [Item kan niet worden toegevoegd]
Beschrijving	1. Het systeem toont een foutboodschap. 2. Het item wordt niet toegevoegd aan het winkelmandje. 3. Terug naar basis scenario stap 1.

9. Use case 9: Bestelgeschiedenis Bekijken

Actor	Super Admin, Admin, Gebruiker
Aannamen	De actor is ingelogd en heeft bestellingen geplaatst.
Beschrijving	1. De actor navigeert naar zijn profile pagina. 2. De actor opent zijn bestelgeschiedenis. 3. Het systeem haalt alle orders op.
Uitzonderingen	3A [Geen bestellingen gevonden]
Resultaat	De actor heeft een inzicht in zijn eerdere transacties.

Alternatieve scenario's

Naam	3A [Geen bestellingen gevonden]
Beschrijving	1. Het systeem toont een foutboodschap. 2. De actor navigeert terug naar de homepage. 3. Terug naar basis scenario stap 1.

10. Use case 10: Gebruikers aanmaken

Actor	Super Admin, Admin
Aannamen	De actor is succesvol ingelogd op het beheerportaal en bevindt zich op het gebruikers overzicht scherm.
Beschrijving	1. De actor klikt op de knop om een nieuwe gebruiker aan te maken. 2. Het systeem toont het aanmaakformulier. 3. De actor vult de juiste gegevens in. 4. De actor selecteert de rol voor de nieuwe gebruiker. 5. De actor selecteert de instantie voor de gebruiker. 6. De actor klikt op de knop om de gebruiker aan te maken. 7. Het systeem valideert de invoer. 8. Het systeem maakt de nieuwe gebruiker aan. 9. Het systeem toont de nieuwe gebruiker in het overzicht.
Uitzonderingen	5A [Geen instantie beschikbaar]
Resultaat	De actor heeft een nieuwe gebruiker kunnen aanmaken en deze kunnen koppelen aan een instantie.

Alternatieve scenario's

Naam	5A [Geen instantie beschikbaar]
Beschrijving	1. Het systeem toont een foutmelding. 2. De actor selecteert nog geen instantie. 3. De actor controleert de gegevens. 4. Terug naar basis scenario stap 6.

C. Functioneel design

In het functionele design van dit project geef ik een gedetailleerd overzicht van de navigatie flow en de interactie tussen de verschillende schermen. De applicatie heeft verschillende rollen met eigen verantwoordelijkheden. Daarom heb ik de visualisatie opgesplitst per gebruiker.

Bij de flows voor de Super Admin en Admin ligt de nadruk expliciet op hun onderscheidende bevoegdheden, in plaats van een generieke navigatie te herhalen.

Tot slot brengen we voor de gebruiker de volledige flow in kaart.

Navigatie Super Admin:

Actor: Super Admin

Flow:

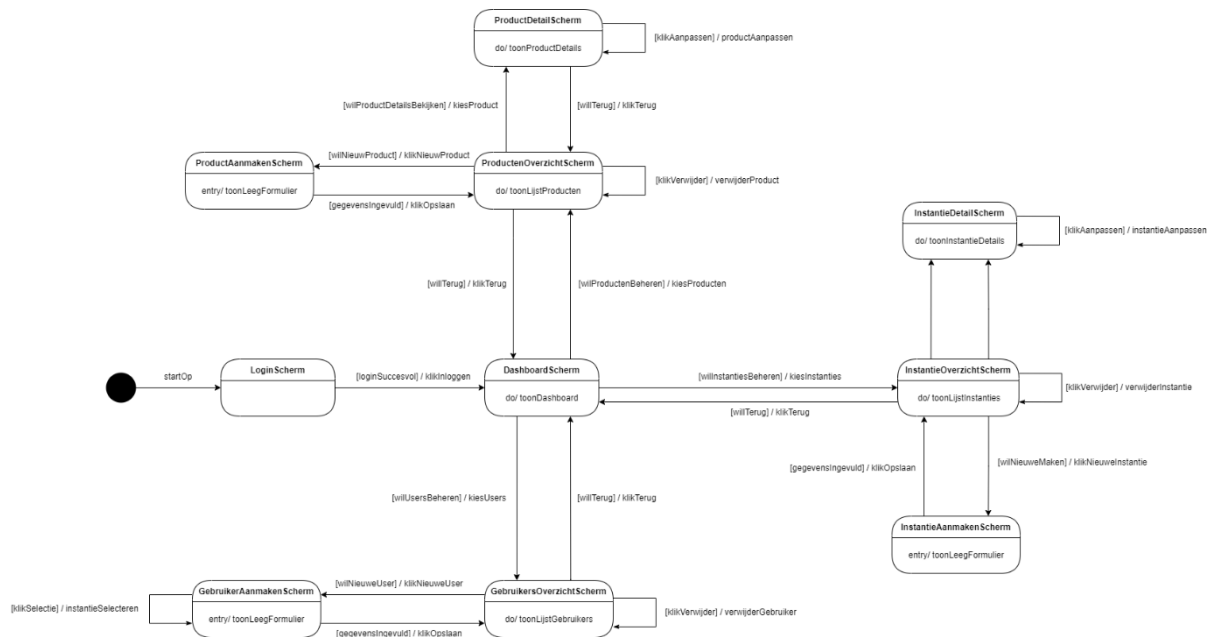
Wanneer de Super Admin de applicatie opent, kan hij navigeren naar het Login Scherm waar hij zijn specifieke gebruikersgegevens moet invullen. Als deze gegevens correct zijn ingevuld, dan wordt hij direct doorgestuurd naar het Dashboard Scherm. Dit scherm dient als een centrale hub van het Admin portaal en biedt toegang tot de drie hoofdpijlers van de infrastructuur: Producten, Instanties en Gebruikers.

Vanuit het dashboard heeft de Super Admin de keuze om te navigeren naar specifieke beheeronderdelen. Kiest hij voor Instanties beheren, dan wordt hij naar het Instantie Overzicht Scherm geleid. Hier ziet hij een lijst van alle actieve instanties. Op dit scherm heeft hij de mogelijkheid om een bestaande omgeving te verwijderen of door te klikken naar het Instantie

Aanmaken Scherm. Hier kan een volledige nieuwe, gescheiden omgeving worden opgezet voor de klant of training.

Daarnaast kan de Super Admin vanaf het dashboard navigeren naar het Producten Overzichten Scherm. Dit geeft een weergave van alle artikelen in de BrightSwagShop. Indien hij een item selecteert, wordt hij doorgestuurd naar het Product Detail Scherm om specifieke details te bekijken of aan te passen. Ook kan hij kiezen om nieuwe Brightest items toe te voegen aan de catalogus via een aanmaakformulier.

Tot slot biedt het dashboard toegang tot het Gebruikers Overzicht Scherm. Hier beheert de Super Admin de toegang voor trainers of externe partijen. Hij kan hier ook nieuwe accounts aanmaken of bestaande gebruikers verwijderen, zodat de juiste personen toegang krijgen tot hun specifieke instantie. Bij elk invulformulier (zoals bij het aanmaken van instanties, producten of gebruikers) kan de Super Admin zijn wijzigingen definitief maken door op Opslaan te klikken, waarna hij automatisch terugkeert naar het overzicht.



Figuur 6. Navigatie model super admin

Navigatie Admin:

Actor: Admin

Flow:

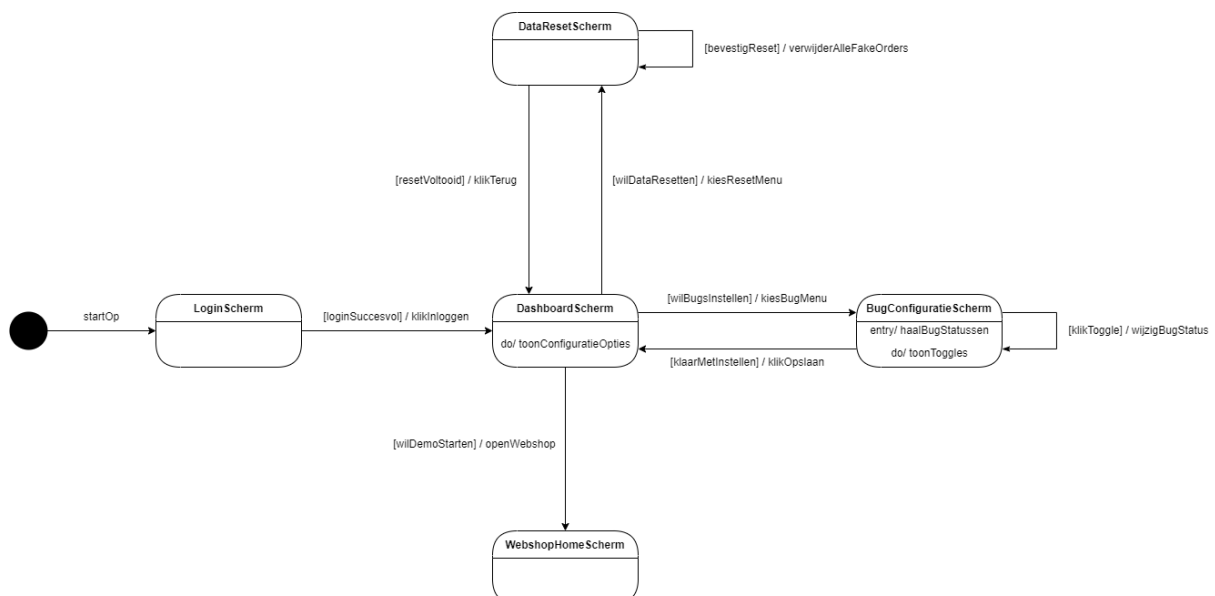
Wanneer de Admin de applicatie opent, kan hij navigeren naar het Login Scherm waar hij zijn specifieke gebruikersgegevens moet invullen. Als deze gegevens correct zijn ingevuld, dan wordt hij direct doorgestuurd naar het Dashboard Scherm.

In tegenstelling tot de Super Admin, die de infrastructuur beheert, dient dit dashboard als het operationele controle scherm voor trainingen en demo's. Het biedt toegang tot drie specifieke pijlers: Data Resetten, Bug configuratie en de Webshop zelf.

Kiest de Admin voor Data Reset, dan navigeert hij naar het Data Reset Scherm. Dit is een essentiële functie voor trainers. Na een sessie zit de database vaak vol met "fake" bestellingen van cursisten. Hier heeft de Admin de mogelijkheid om met één klik al deze testdata te verwijderen. Zo wordt de omgeving opgeschoond en klaargezet voor de volgende groep, zonder dat er verwarring ontstaat door de oude gegevens.

De kernfunctionaliteit van de applicatie bevindt zich op het Bug Configuratie Scherm. Hier haalt het systeem de huidige status van de bugs op. De Admin ziet een lijst met toggles waarmee specifieke bugs aan- en uitgezet kunnen worden. Zodra hij op Opslaan drukt, keert hij terug naar het dashboard en zijn bugs zijn actief.

Tot slot biedt het dashboard een directe navigatie naar het Webshop Scherm. Hiermee schakelt de Admin over van de beheerdermodus naar gebruikersmodus. Dit stelt de trainer in staat om de zojuist geconfigureerde bugs direct te demonstreren aan het publiek.



Figuur 7. Navigatie model admin

Navigatie Gebruiker:

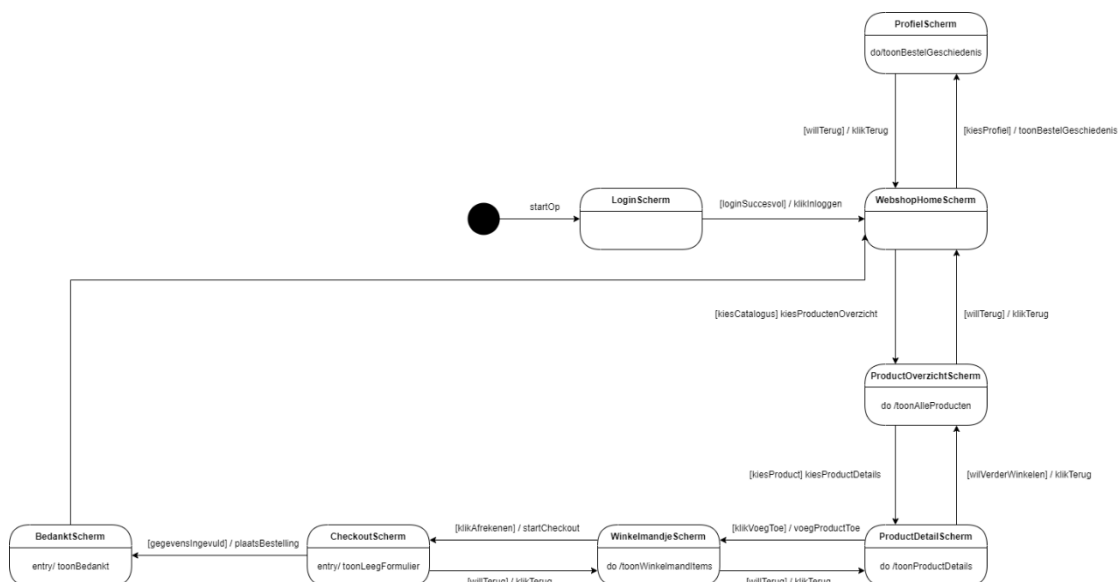
Actor: Gebruiker

Flow:

Wanneer de Gebruiker de applicatie opent en inlogt via het Login Schem, wordt hij eerst doorverwezen naar het Instantie Selectie Schem. Omdat de applicatie met gescheiden omgevingen werkt, moet de gebruiker hier expliciet kiezen met welke trainingen of demo-omgeving hij wil verbinden. Pas na deze keuze opent het Webshop Home Schem.

Vanuit hier start het daadwerkelijke winkel- en testproces. De gebruiker kan door de catalogus navigeren via het Product Overzicht naar het Product Detail Schem. Hier kunnen items aan het winkelmandje worden toegevoegd om vervolgens de bestelflow te starten. Via het Checkout Schem vult de gebruiker zijn gegevens in en rondt hij de bestelling af, waarna het Bedankt Schem de bevestiging toont.

Tot slot biedt het Profiel Schem toegang tot de bestelgeschiedenis en adresgegevens.



Figuur 8. Navigatie model gebruiker

1. Scherm 1: Inloggen

Use case(s): Login, Wachtwoord resetten

Invoerveld gebruikersnaam: veld waarin de gebruiker het e-mailadres dient in te voeren.

Invoerveld wachtwoord: veld voor het invoeren van het wachtwoord.

Knop “Login”: actieknop om de gegevens te verifiëren. Bij succes wordt de gebruiker direct doorgestuurd naar de homepagina.

The figure consists of two side-by-side screenshots of a login interface for 'BrightSwagShop'.
The left screenshot shows a vertical layout with the text 'welcome to brightswagshop' in a stylized font, followed by 'please login or register to continue'. Below this are two input fields: 'Username' (with a person icon) and 'Password' (with a lock icon). Below the password field is a link 'forgot password? [click here](#)'. At the bottom of this section are two buttons: 'Continue' and 'Register'. At the very bottom is a large logo consisting of a yellow circle with a white stylized 'b' inside.
The right screenshot shows a white card on a light gray background. The card has the title 'Log in op uw account' and a subtitle 'Vul uw gegevens hieronder in om in te loggen.' Below the subtitle are two input fields: 'Username' (with a person icon) and 'Password' (with a lock icon). Below the password field is a yellow 'Login' button. At the bottom of the card is a link 'Nog geen account? [Register](#)'.

Figuur 9. Scherm login

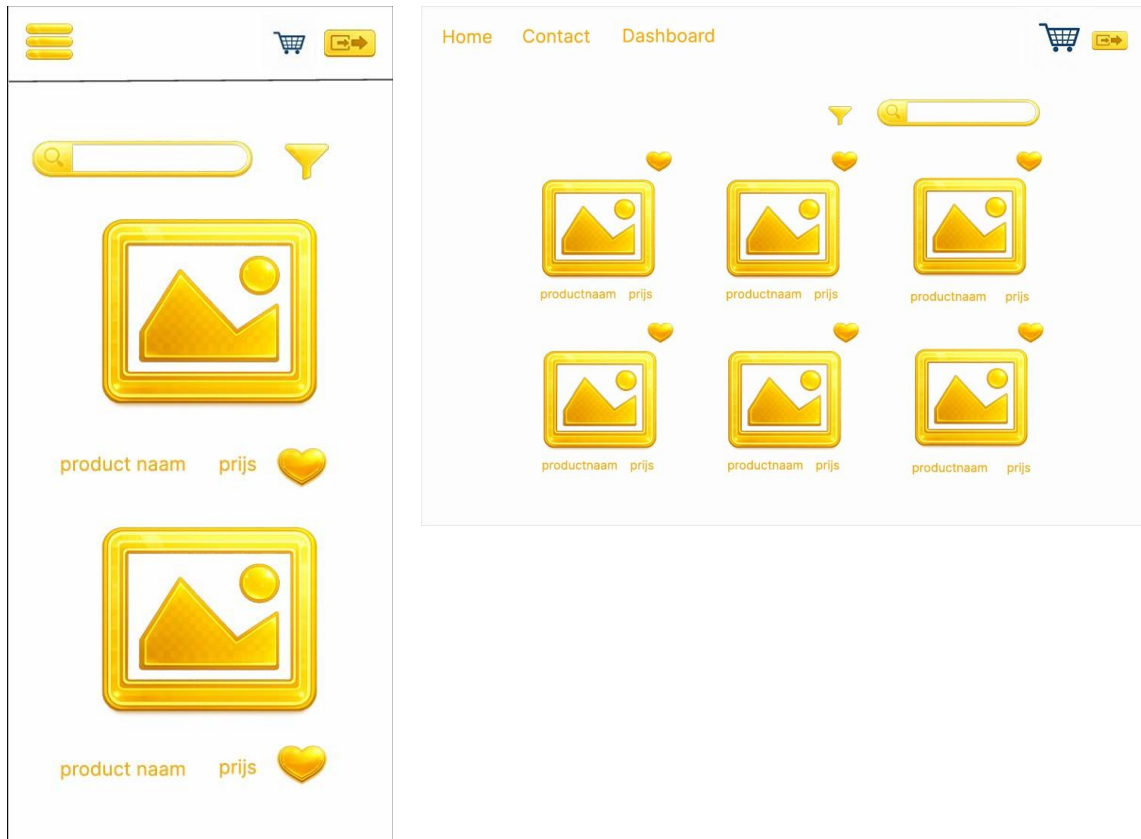
2. Scherm 2: Home

Use case(s): Producten raadplegen, Zoeken & Filteren

Items: Een volledig overzicht van het beschikbare assortiment in de webshop.

Filter knop: Naast de zoekbalk bevindt zich een filter knop, hiermee kan de gebruiker filteren op verschillende filter criteria.

Zoekbalk: Zoekfunctionaliteit om specifieke producten binnen de shop te vinden.



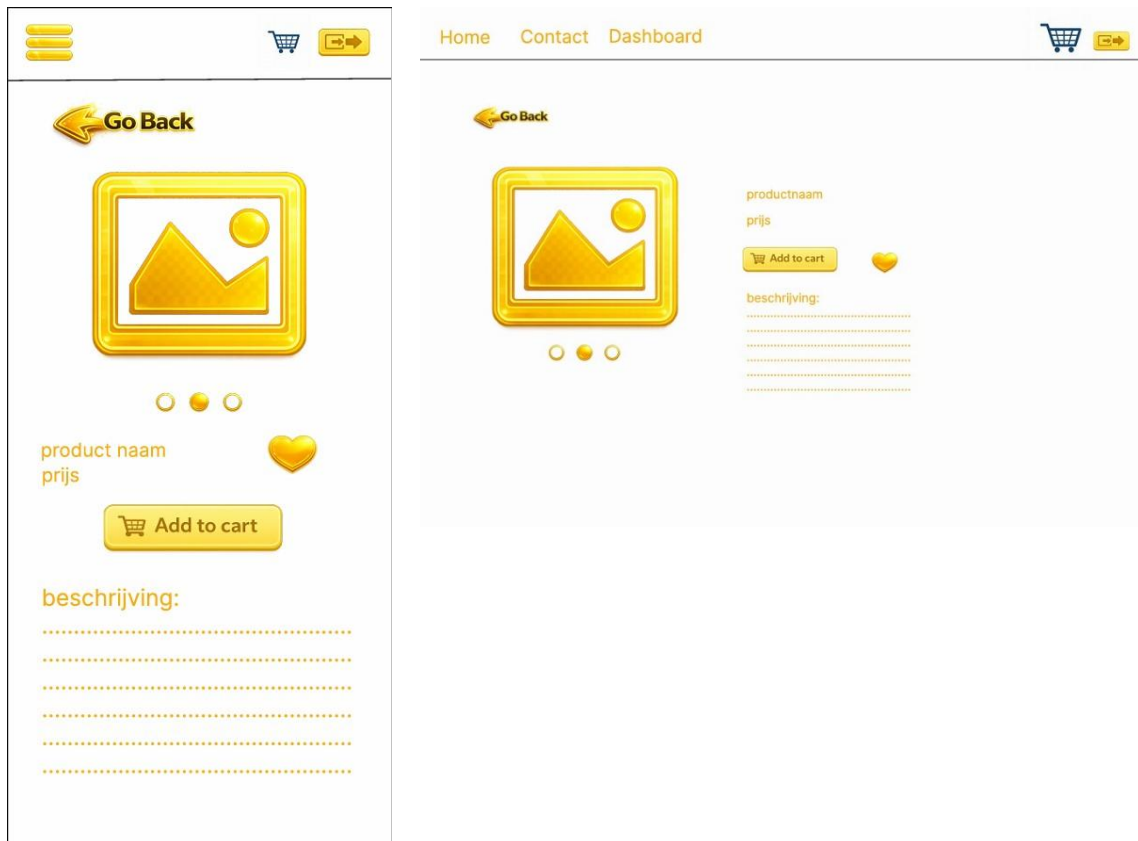
Figuur 10. Scherm home

3. Scherm 3: Product details

Use case(s): Product details bekijken

Item: weergave van product specifieke informatie, waaronder de naam, prijs en omschrijving.

Knop “In winkelwagen”: functionaliteit waarmee de gebruiker het geselecteerde product aan de winkelwagen toevoegt.



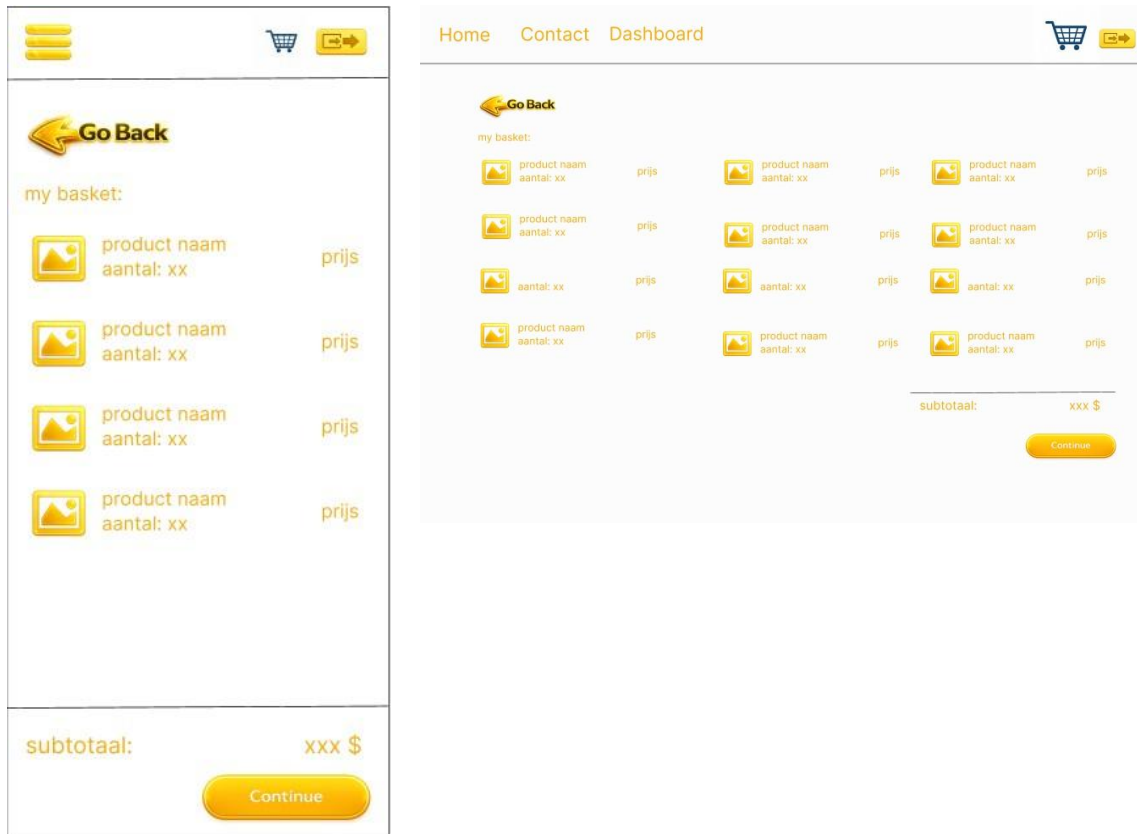
Figuur 11. Scherm product details

4. Scherm 4: Winkelmandje

Use case(s): Winkelmandje beheren

Overzicht: een volledige lijst van alle artikelen die momenteel in de winkelwagen zijn geplaatst.

Totaal: Weergave van de totaalprijs van alle geselecteerde producten.



Figuur 12. Scherm winkelmandje

5. Scherm 5: Bestelling afronden

Use case(s): Bestelling plaatsen, Bestelling simuleren

Invoervelden: formulier voor de invoer van klantgegevens, waaronder voornaam, achternaam, e-mailadres en adresgegevens.

Knop “Verder”: na bevestiging simuleert het systeem de betalingstransactie en toont vervolgens een melding dat de bestelling succesvol is verwerkt.

The image displays two versions of a checkout screen. The left version is a mobile interface with a hamburger menu icon, a shopping cart icon, and a 'checkout:' heading. The right version is a desktop interface with navigation links (Home, Contact, Dashboard), a 'Go Back' button, and the same checkout form. Both versions feature a form with fields for Name, Surname, E-mail, and Address, a 'Verder' button, and payment options (Apple Pay, Klarna, PayPal).

Mobile View (Left):

checkout:

Naam
Jouw naam

Achternaam
Jouw achternaam

E-mail
naam@voorbeeld.com

Adres
Straat + huisnummer

Verder

Apple Pay Klarna. PayPal

Desktop View (Right):

Home Contact Dashboard

Go Back

checkout:

Naam
Jouw naam

Achternaam
Jouw achternaam

E-mail
naam@voorbeeld.com

Adres
Straat + huisnummer

Verder

Apple Pay Klarna. PayPal

Figuur 13. Scherm Bestelling afronden

6. Scherm 6: Contact

Use case(s): geen.

Omschrijving: deze pagina bevat geen specifieke functionaliteiten en fungeert als opvulling binnen de applicatie.

The image displays two mobile application screens for the contact page. Both screens feature a top navigation bar with a hamburger menu icon, a shopping cart icon, and a right arrow icon. The left screen has a 'contact:' header and a 'Contactformulier' box with fields for 'Naam' (Name), 'E-mail', and 'Bericht' (Message), and a 'Verstuur' (Send) button. The right screen has a 'Home Contact Dashboard' header, a 'contact:' label, and a 'Contactformulier' box with fields for 'Naam', 'E-mail', and 'Bericht', and a 'Verstuur' button. It also includes 'Follow us' and 'Find us' sections with social media icons and a location pin.

Left Screen:

contact:

Contactformulier

Naam
Jouw naam

E-mail
naam@voorbeeld.com

Bericht
Typ hier je bericht...

Verstuur

Right Screen:

Home Contact Dashboard

contact:

Contactformulier

Naam
Jouw naam

E-mail
naam@voorbeeld.com

Bericht
Typ hier je bericht...

Verstuur

Follow us

Facebook, Twitter, Instagram, LinkedIn

Find us

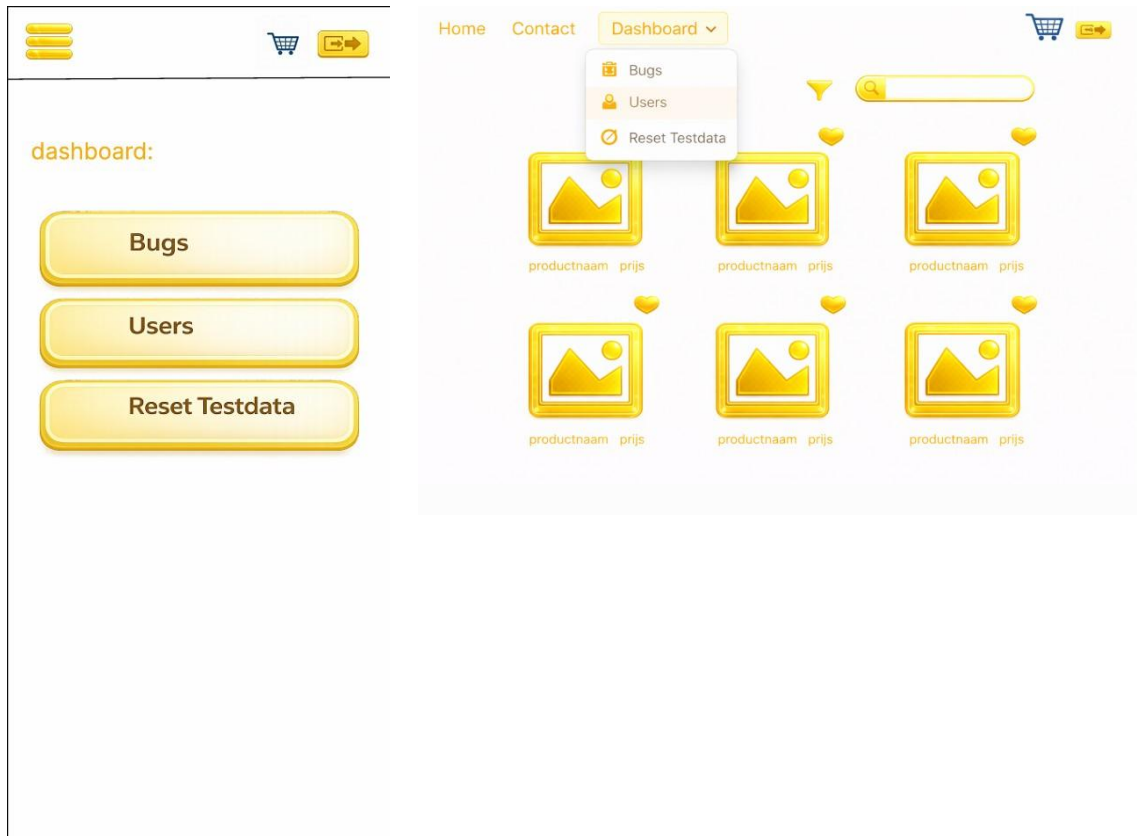
1234 Street Name
City, State ZIP

Figuur 14. Scherm contact

7. Scherm 7: Admin dashboard

Use case(s): Dashboard raadplegen

Navigatie knoppen: het dashboard fungeert als centraal startpunt voor de Admin. Via navigatie knoppen is er direct toegang tot Bugsconfiguratie, Gebruikersbeheer en het Testdataresetscherm.

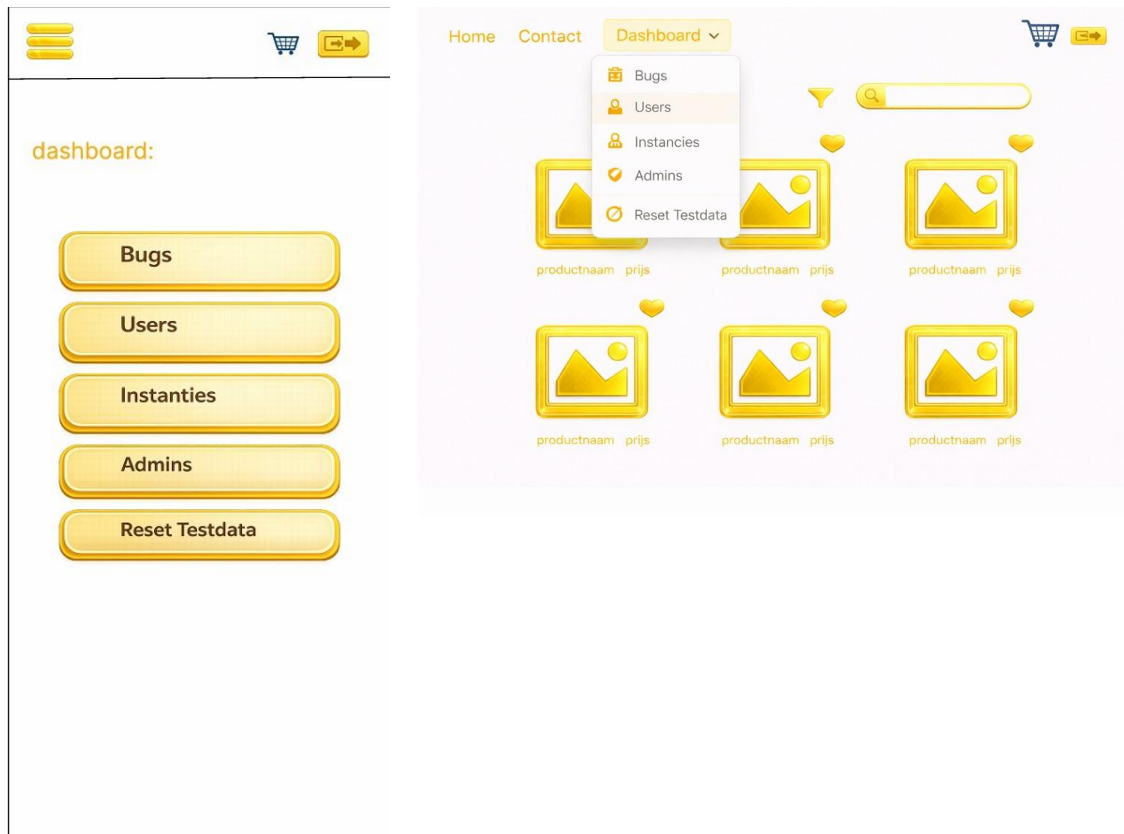


Figuur 15. Scherm admin dashboard

8. Scherm 8: Super Admin dashboard

Use case(s): Dashboard raadplegen

Navigatie knoppen: het dashboard fungeert als centraal startpunt voor de Super Admin. Via navigatie knoppen is er direct toegang tot Bugsconfiguratie, Gebruikersbeheer, Instantiebeheer, Adminbeheer en het Testdataresetscherm.



Figuur 16. Scherm super admin dashboard

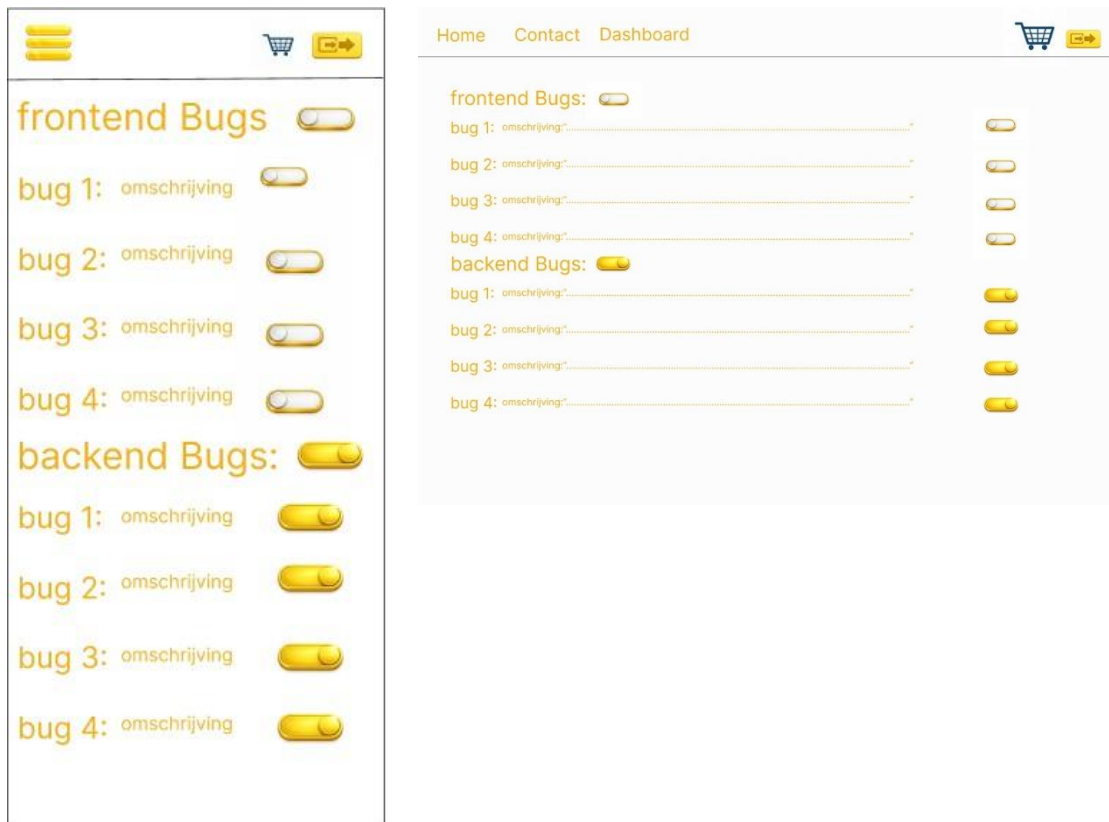
9. Scherm 9: Bug configuratie

Use case(s): Bugs configureren

Frontend bugs: functionaliteit voor de Super Admin en Admin om frontend gerelateerde bugs individueel aan of uit te schakelen. Tevens is er een “Alles activeren” knop beschikbaar om alle bugs in deze categorie gelijktijdig te wijzigen.

Backend bugs: functionaliteit voor de Super Admin en Admin om backend gerelateerde bugs individueel aan of uit te schakelen. Tevens is er een “Alles activeren” en “Alles deactiveren” knop beschikbaar om alle bugs in deze categorie gelijktijdig te wijzigen.

Details: bekijk alle details per bug. Het overzicht toont de naam, een omschrijving van wat de bug doet en of deze momenteel actief is.



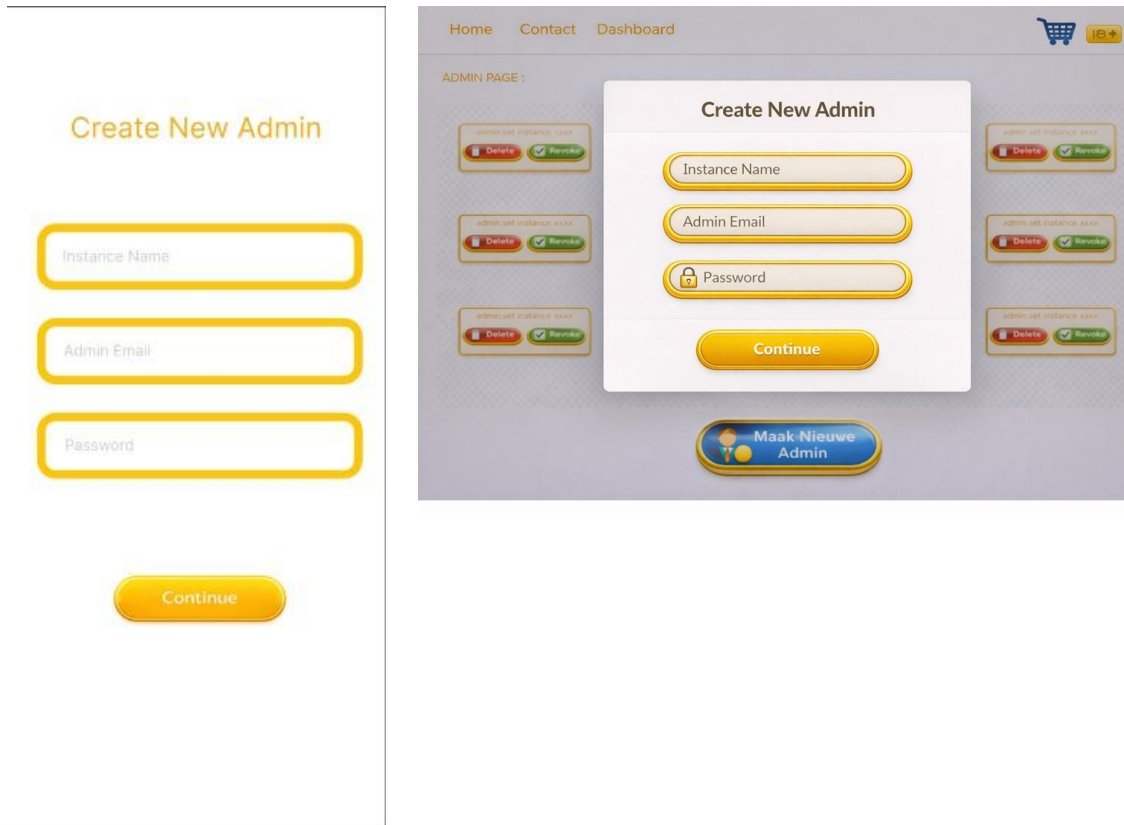
Figuur 17. Scherm bug configuratie

10. Scherm 10: Admin aanmaken

Use case(s): Admin aanmaken, Instantie selecteren

Involformulier: formulier voor het invoeren van de gegevens van een nieuwe Admin en het selecteren van de bijhorende instantie.

Knop “Aanmaken”: klik op de knop om een nieuwe Admin definitief aan te maken.



The image displays two versions of the 'Create New Admin' interface. The left version is a clean, minimalist form with three yellow-bordered input fields labeled 'Instance Name', 'Admin Email', and 'Password'. A single yellow 'Continue' button is positioned at the bottom. The right version shows this form as a modal window overlaid on a dashboard. The dashboard header includes 'Home', 'Contact', and 'Dashboard' links, along with a shopping cart icon and a user profile icon. The modal window has a title 'Create New Admin' and contains the same three input fields and a yellow 'Continue' button. Below the modal, a blue button with a lightbulb icon and the text 'Maak Nieuwe Admin' is visible.

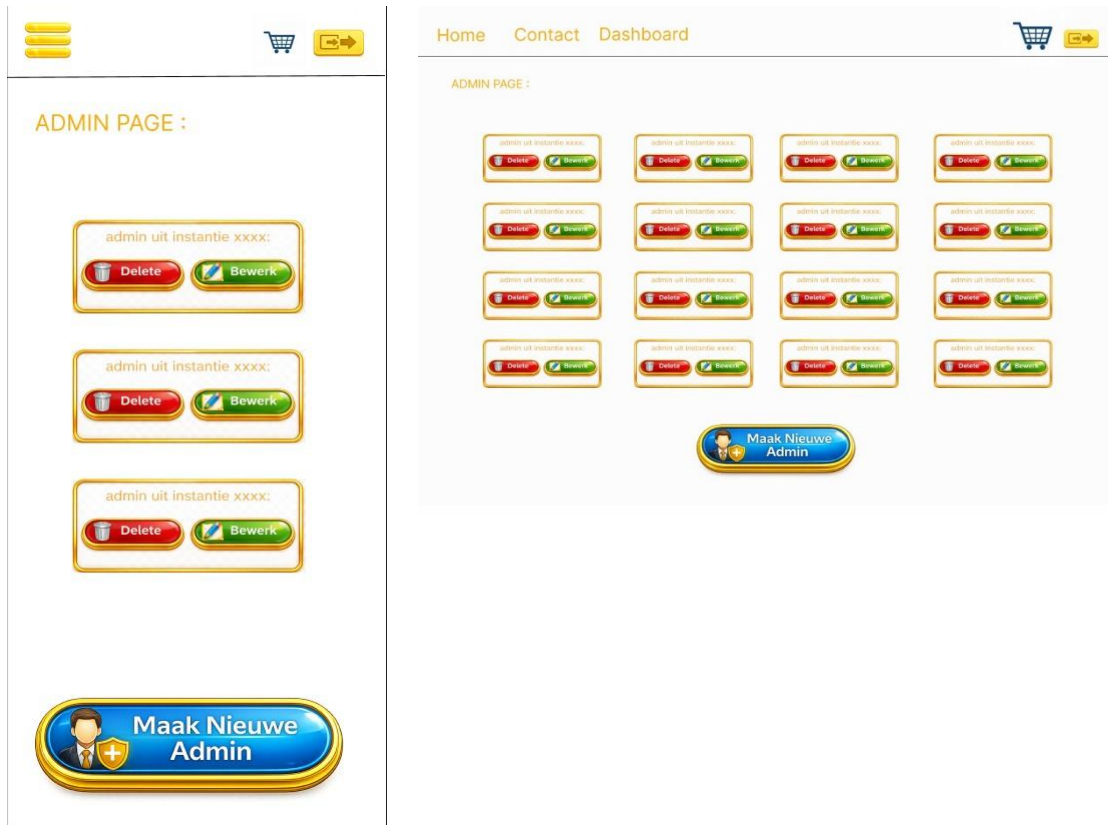
Figuur 18. Scherm admin aanmaken

11. Scherm 11: Admin Beheer

Use case(s): Admins raadplegen, Admin verwijderen, Admin bewerken

Overzicht: een lijstweergave van alle geregistreerde Admins in het systeem, inclusief de instantie waaraan zij zijn gekoppeld.

Beheer: functionaliteit voor de Super Admin om de gegevens van een Admin te wijzigen of het account te verwijderen.

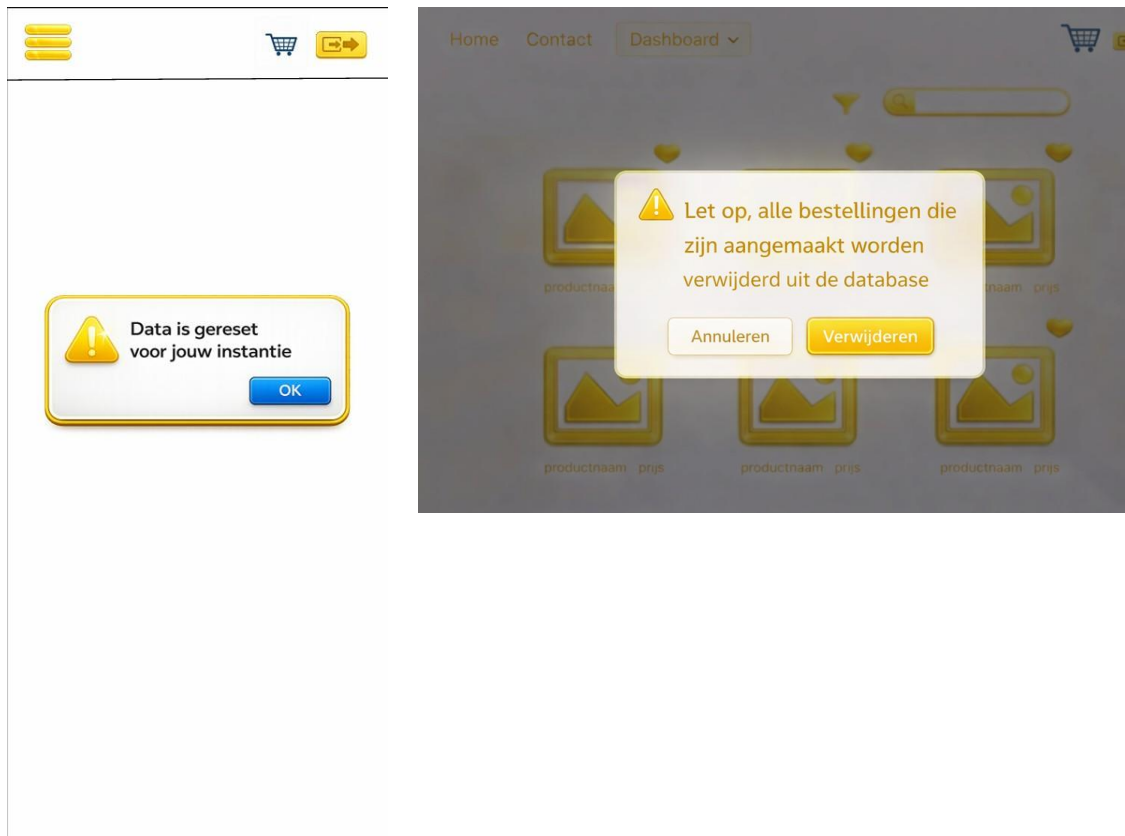


Figuur 19. Scherm admin Beheer

12. Scherm 12: Data reset

Use case(s): Testdata resetten

Data reset knop: functionaliteit waarmee de Admin en Super Admin testgegevens uit het systeem kunnen verwijderen.



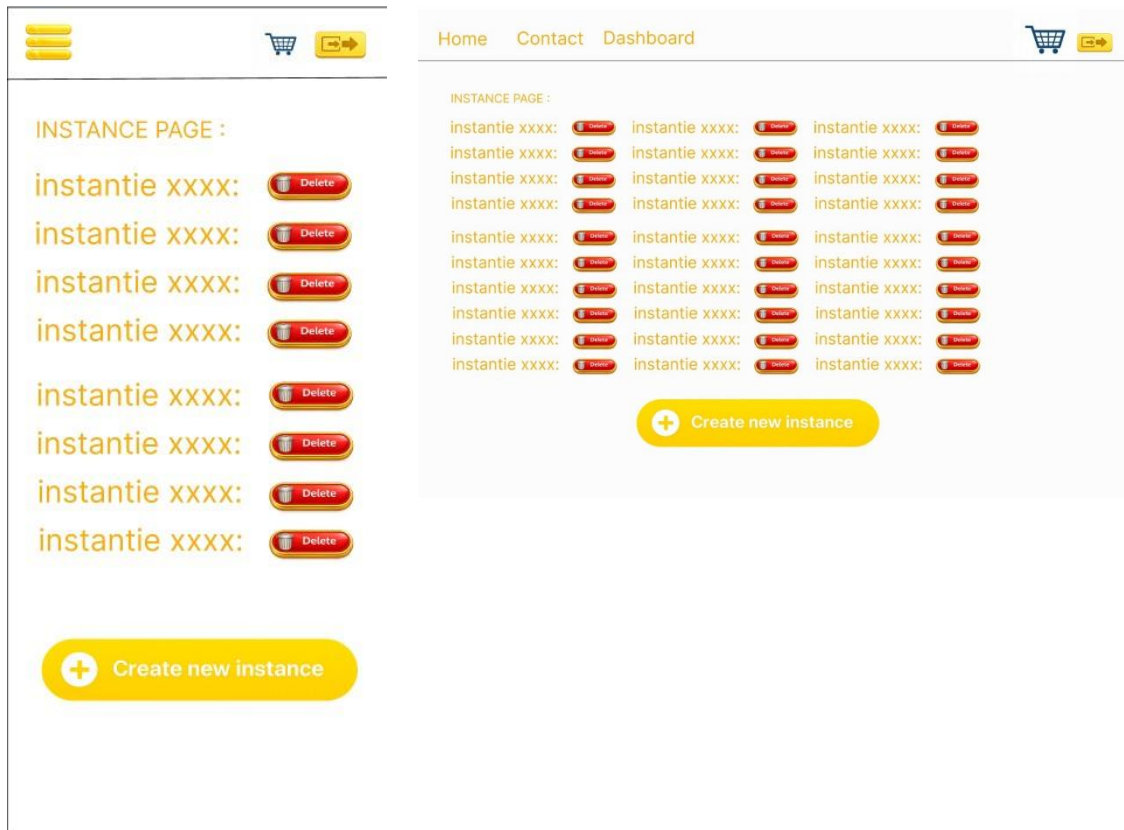
Figuur 20. Scherm data reset

13. Scherm 13: Instantie overzicht Super Admin

Use case(s): Instantie raadplegen, Instantie verwijderen.

Overzicht: een volledige lijst van alle geregistreerde instanties binnen het systeem.

Knop “Verwijderen”: functionaliteit voor de Super Admin om een geselecteerde instantie permanent te verwijderen.



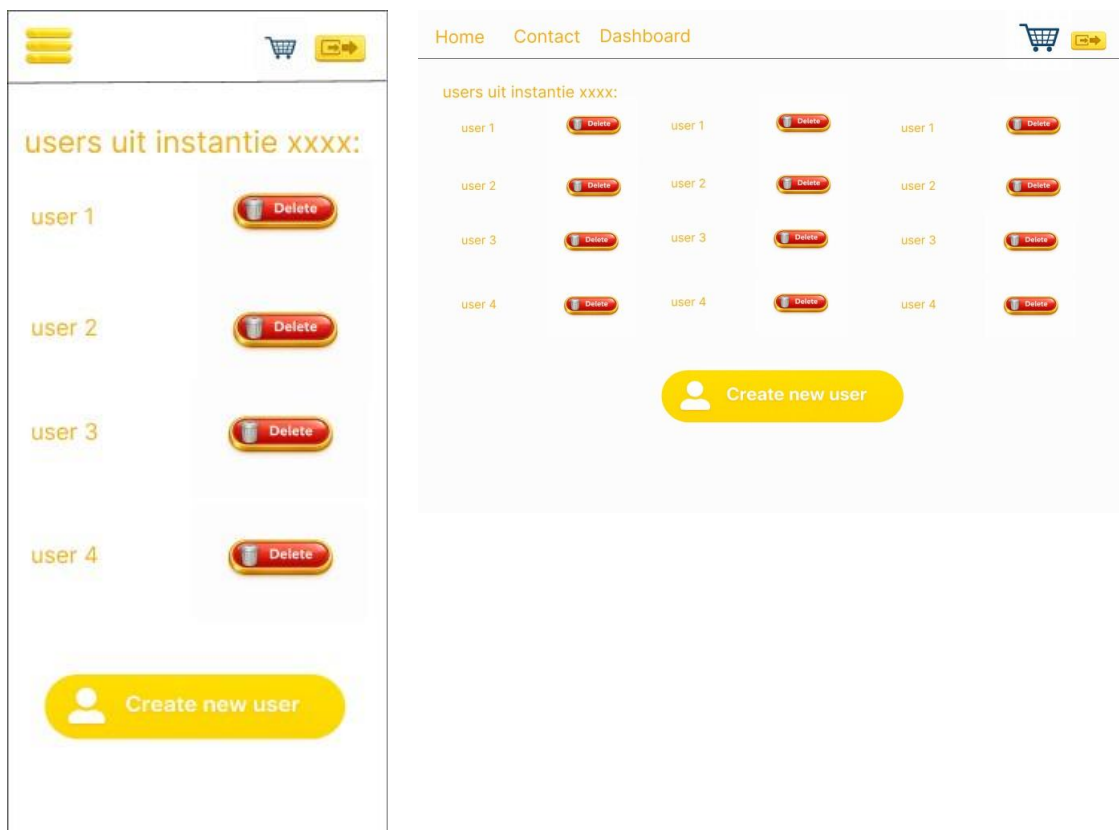
Figuur 21. Scherm instantie overzicht super admin

14. Scherm 14: User overzicht per instantie Admin

Use case(s): Gebruikers raadplegen, Gebruiker verwijderen

Overzicht: een gefilterde lijst van alle gebruikers die gekoppeld zijn aan de specifieke instantie van de Admin.

Knop “Verwijderen”: functionaliteit voor de Admin om een gebruikersaccount uit de eigen instantie te verwijderen.



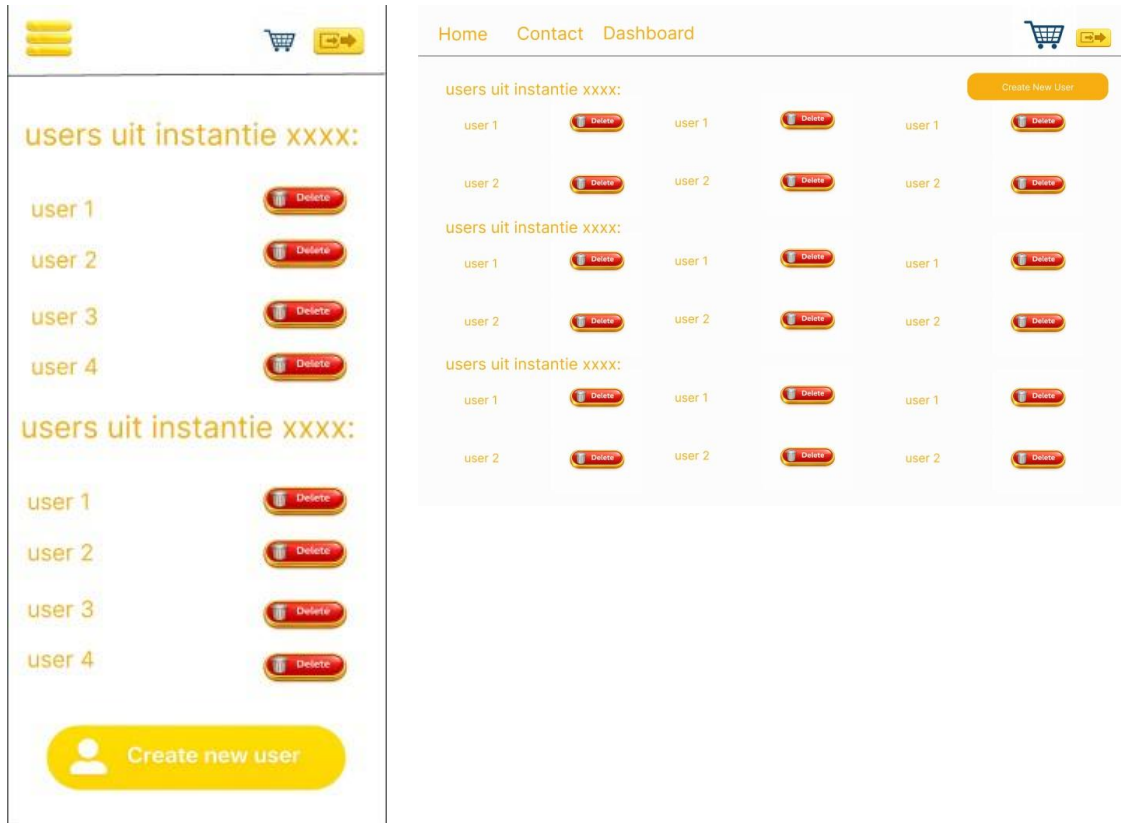
Figuur 22. Scherm user overzicht per instantie admin

15. Scherm 15: User overzicht per instantie Super Admin

Use case(s): Gebruikers raadplegen, Gebruikers verwijderen

Overzicht: een globaal overzicht van alle geregistreerde gebruikers over alle instanties heen.

Knop “Verwijderen”: functionaliteit voor de Super Admin om een gebruikersaccount van een specifieke instantie te verwijderen.



Figuur 23. Scherm user overzicht per instantie super admin

16. Scherm 16: Gebruiker aanmaken pop-up

Use case(s): Gebruikers aanmaken

Pop-up: een dialoogvenster met een invulformulier waarmee de Super Admin en Admin een nieuwe gebruiker in het systeem kan registreren.

The image displays two screenshots of a user creation interface. The left screenshot shows a 'create new user:' screen with a yellow header. It features three input fields: 'Username' (with a person icon), 'Password' (with a lock icon), and a dropdown menu labeled 'Kies een instance' with a list of instances (Instance 1 through Instance 5). A yellow 'Continue' button is at the bottom. The right screenshot shows a pop-up dialog box over a background of a user list. The dialog has a white background and contains three input fields: 'Username' (with placeholder 'Enter username'), 'Password' (with placeholder 'Enter password'), and 'Repeat password' (with placeholder 'Repeat password'). At the bottom of the dialog are two buttons: 'Annuleren' (grey) and 'Bevestigen' (yellow). The background shows a table of users with columns for 'users uit instantie xxxx', 'user 1', 'user 2', 'user 3', and 'user 4', and a 'Delete' button for each row.

Figuur 24. Scherm gebruiker aanmaken pop-up

17. Scherm 17: Instantie aanmaken pop-up

Use case(s): Nieuwe instantie aanmaken

Pop-up: een dialoogvenster met een invulformulier waarmee de Super Admin een nieuwe instantie kan aanmaken.

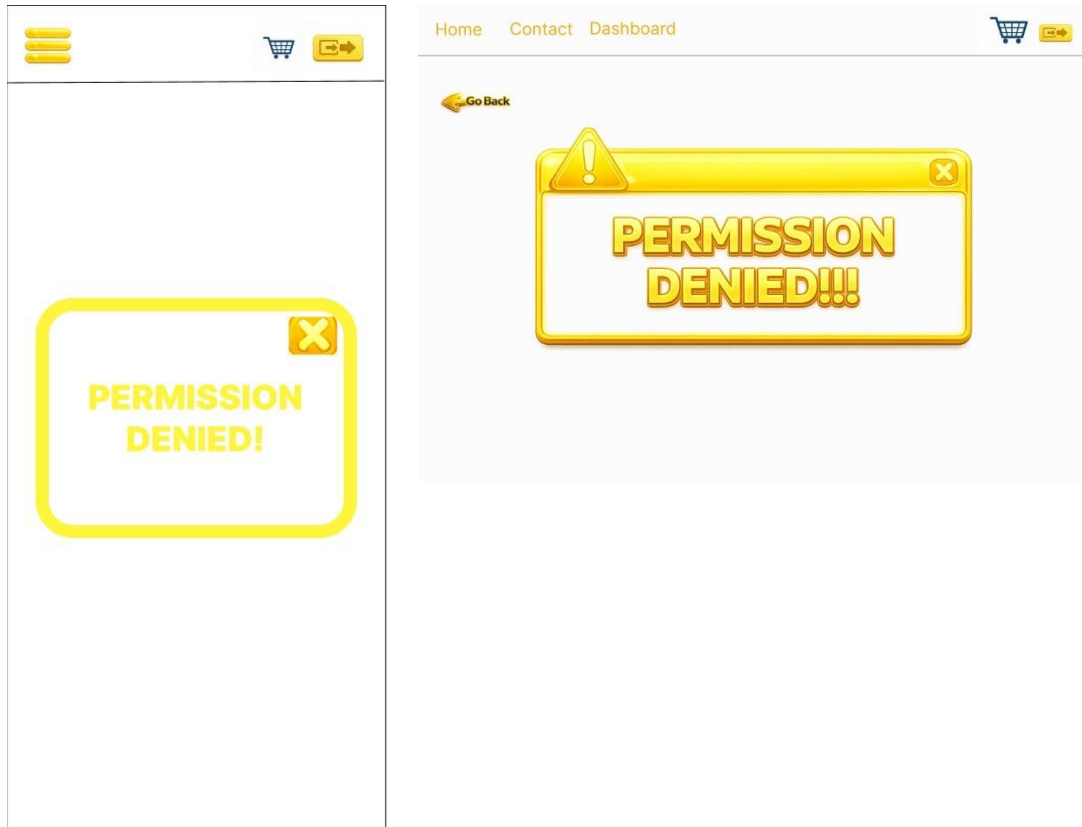
The image displays two versions of a 'create new instance' pop-up dialog. The left version is set against a light gray background and features the title 'create new intstance:' in orange text at the top. The right version is set against a dark gray background. Both dialogs are white with rounded corners and contain the following elements: a label 'Instance name' above a text input field with the placeholder 'Enter instance name', and two buttons at the bottom: a light gray 'Annuleren' button and a yellow 'Bevestigen' button.

Figuur 25. Scherm instantie aanmaken pop-up

18. Scherm 18: Geen toegang pop-up

Use case(s): geen.

Omschrijving: deze pagina fungeert als foutmelding wanneer een gebruiker probeert te navigeren naar een pagina waarvoor hij onvoldoende rechten voor heeft.



Figuur 26. Scherm geen toegang pop-up

19. Scherm 19: Instantie details

Use case(s): Instantie details bekijken, Instantie bijwerken

Overzicht: weergave van gedetailleerde informatie over de geselecteerde instantie.

Knop “Bewerken”: functionaliteit waarmee de Super Admin de gegevens van de huidige instantie kan wijzigen.

The image displays two wireframe versions of the 'Instantie details' screen. The left version is a mobile layout, and the right version is a desktop layout. Both screens show the following elements:

- Header:** Mobile layout includes a hamburger menu, shopping cart, and edit icon. Desktop layout includes navigation links (Home, Contact, Dashboard) and the same icons.
- Form Fields:** 'Instantie naam', 'Instantie xxxx', and 'Instantie beschrijving' (with a text area).
- Action:** A 'Bewerken' (Edit) button.

Figuur 27. Scherm instantie details

20. Scherm 20: Register

Use case(s): Register

Invoerveld gebruikersnaam: veld waarin de gebruiker het e-mailadres dient in te voeren.

Invoerveld wachtwoord: veld voor het invoeren van het wachtwoord.

Instantie selectie: hier kan de gebruiker zijn gewenste instantie selecteren.

Knop “Register”: actieknop om de gegevens te verifiëren. Bij succes wordt de gebruiker aangemaakt en direct doorgestuurd naar de homepagina.

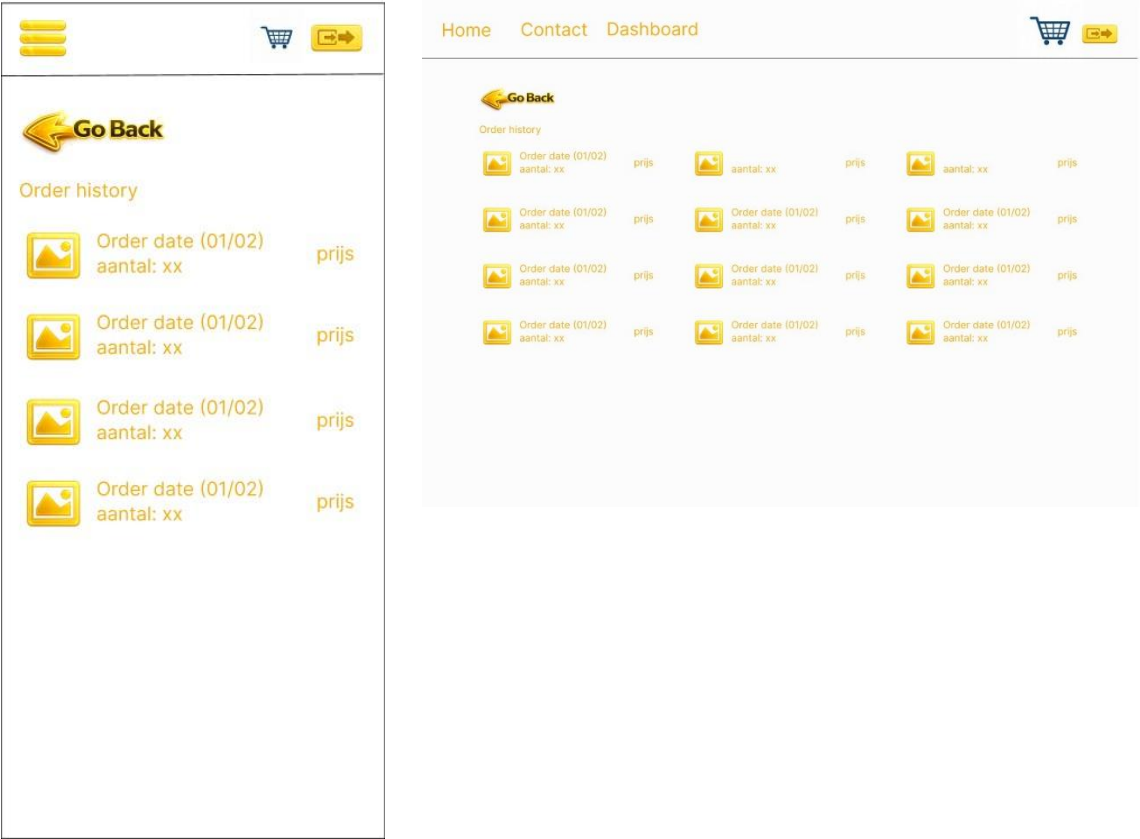
The figure displays two versions of a registration interface. The left version is a vertical form with a yellow 'Username' field, a yellow 'Password' field, and a dropdown menu for 'Kies een instance' with options 'Instance 1' through 'Instance 5'. Below these is a yellow 'Register' button and a large yellow logo. The right version is a horizontal form titled 'Maak een nieuw account aan' with a subtitle 'Vul de gegevens hieronder in om een account aan te maken.' It features a 'Username' field, a 'Password' field, and a 'Kies een instance' dropdown with the same options. A yellow 'Continue' button is positioned below the fields. Both versions use a yellow and white color scheme.

Figuur 28. Scherm register

21. Scherm 21: Bestelling geschiedenis

Use case(s): Bestelgeschiedenis bekijken

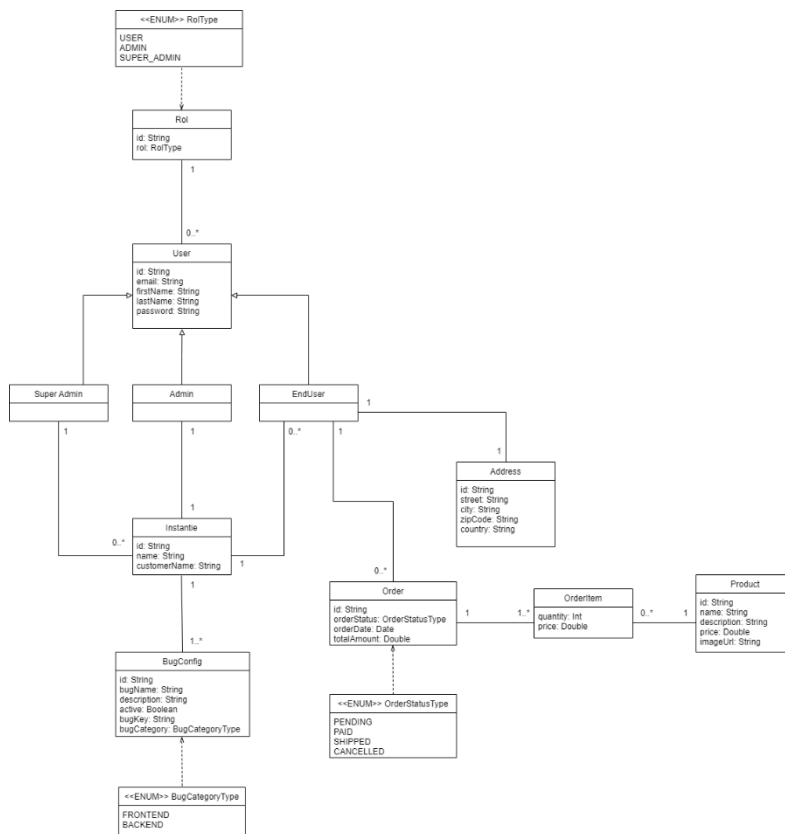
Overzicht: een volledige lijst van alle bestellingen die je hebt geplaatst met details zoals de besteldatum en totaalprijs van de bestelling.



Figuur 29. Scherm bestelling geschiedenis

D. Informatie

Voor het visualiseren van de informatiearchitectuur is er gekozen om een Klasse diagram uit te werken in plaats van een Data Flow Diagram. De reden hiervoor is dat de complexiteit van de BrightSwagShop niet zit in de beweging van de data, maar in de hiërarchische structuur en de strikte scheiding van gegevens tussen de verschillende trainingen, demo's en recruiting.



Figuur 30. KD BrightSwagShop

Het bovenstaand klassendiagram biedt een overzicht van de belangrijkste klassen en hun onderlinge relaties binnen het systeem. In het diagram is de structuur gevisualiseerd, waarbij duidelijk wordt hoe entiteiten zoals gebruikers, instanties en orders technisch aan elkaar gekoppeld zijn. De relaties verduidelijken de hiërarchie binnen de applicatie en tonen aan welke data bij welke actor hoort.

E. Technisch design

a) Architectuur

De BrightSwagShop is ontworpen als een gedistribueerde applicatie. Om de ontwikkelingssnelheid te waarborgen, zijn de frontend en backend gescheiden in twee aparte code repositories. Dit stelt ons in staat om wijzigingen aan de API door te voeren zonder direct impact te hebben op de gebruikersinterface.

Backend Architectuur (Layered Architecture)

Voor de backend hanteren we een strikte gelaagde architectuur. Dit patroon zorgt voor een duidelijke scheiding van verantwoordelijkheden, we maken gebruik van de volgende lagen: Controller Layer, Service Layer en Data Access Layer.

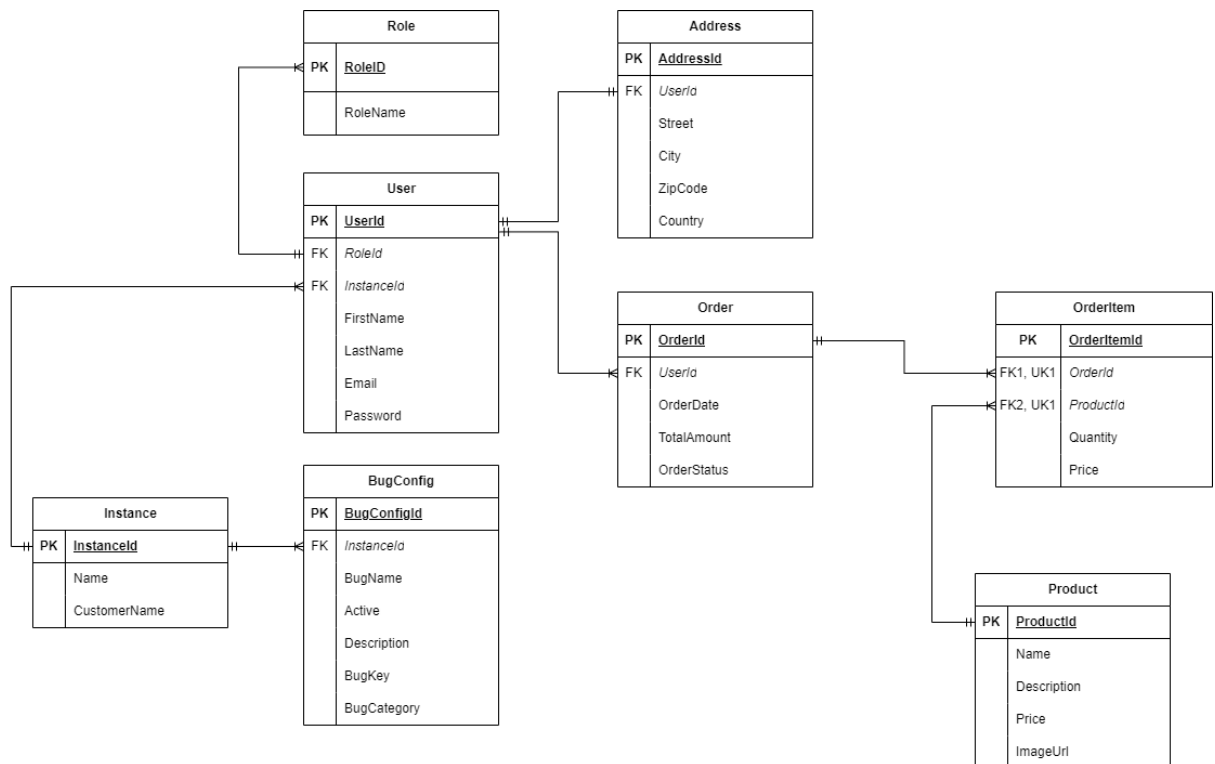
CI/CD & Testing Strategie

Omdat de code in aparte repositories leeft, hebben we ook gescheiden pijplijnen voor kwaliteitsbewaking.

- **Backend pijplijn:**
 - **Basis:** bij elke wijziging draaien de unit tests om de logica in de service laag te verifiëren.
 - **Uitbreiding:** indien de scope het toelaat, worden hier integratietests aan toegevoegd om te controleren of de API endpoints de verwachte statuscodes en data teruggeven.
- **Frontend pijplijn:**
 - **Basis:** bij elke push wordt gecontroleerd of de applicatie succesvol kan bouwen.
 - **Uitbreiding:** als aanvulling kunnen geautomatiseerde tests op de user flow worden toegevoegd om te garanderen dat de interface functioneert volgens de verwachtingen.

Onderstaand Entity Relationship Diagram geeft een overzicht van de gegevens die we gaan bijhouden voor het systeem. Het diagram toont de relaties tussen de entiteiten zoals gebruikers, bug configuraties, instanties, bestellingen en adressen. Hier kunnen we dus zien hoe deze entiteiten met elkaar in verband zijn en op welke manier de gegevens worden beheerd.

In het klassendiagram is er sprake van overerving bij de User entiteiten. Voor de database implementatie is er gekozen voor de single tabel strategie. Dit betekent dat alle gebruiker types in één tabel User worden opgeslagen, waarbij de kolom Role als discriminator fungeert om het type gebruiker aan te duiden.



Figuur 31. ERD BrightSwagShop

b) Integraties

Binnen de architectuur van BrightSwagShop is ervoor gekozen om de applicatie volledig self-contained te houden. Om die reden bevat de applicatie geen externe integraties of afhankelijkheden met systemen van derden. Alles wordt intern afgehandeld of gesimuleerd om de architectuur gesloten en onafhankelijk te houden.

Mock Payment Provider

Hoewel de applicatie een webshop is, wordt er geen daadwerkelijke integratie gebouwd met betaalproviders zoals Stripe, PayPal of Klarna. De “betaling” wordt intern gesimuleerd door de backend. De input wordt lokaal gevalideerd op formaat, maar er vindt geen data uitwisseling plaats met externe banken.

- **Input:** De frontend stuurt de ingevoerde betaalgegevens zoals kaartnummer en de vervaldatum naar de backend voor lokale validatie.
- **Output:** De backend genereert en retourneert transactiestatus en een fictief transactie-ID om het resultaat te simuleren.

c) Datamigratie

Aangezien de BrightSwagShop wordt ontwikkeld als een volledig nieuwe applicatie, is er geen sprake van datamigratie vanuit een bestaand legacy systeem of verouderde database. De database wordt volledig van nul opgezet volgens het nieuwe datamodel. Hierdoor zijn complexe datamigratie technieken niet van toepassing.

Data Cleansing en Enrichment:

Hoewel er geen externe data wordt gemigreerd, start de applicatie niet met een lege database. Omdat de applicatie dient als testobject voor demo's, trainingen en recruiting, is het belangrijk dat elke nieuwe instantie gevuld is met bruikbare data. Dit proces verloopt via database seeding.

Geen data cleansing nodig: omdat we de data niet importeren uit vervuilde bronnen, maar zelf genereren via scripts, is de datakwaliteit bij voorbaat gegarandeerd.

Data Enrichment: tijdens het seeding proces wordt de database verrijkt met specifieke scenario's die nodig zijn voor testdoeleinden.

- **Productcatalogus:** er wordt een standaardlijst van Brightest producten ingeladen zodat de shop niet leeg is.
- **Gebruikersrollen:** standaard account voor Super Admin, Admin en Gebruikers worden aangemaakt.
- **Bug Scenario's:** specifieke data configuraties die nodig zijn om opzettelijke bugs te activeren, worden als startwaarden ingeladen.

d) Technologie

Hieronder vind je een overzicht van de technologieën die zullen worden ingezet voor het project, samen met de argumenten die onze keuze voor deze technologieën onderbouwen.

Frontend technologieën:

- **Next.js**
 - **Beschrijving:** Next.js is een open-source React framework dat extra functionaliteiten toevoegt zoals server-side rendering en statische website generatie. Het is ontworpen om de prestaties, file based routing, SEO en de ontwikkelervaring van webapplicaties te optimaliseren.
 - **Argumenten:**
 - **Typescript integratie:** Next.js werkt naadloos samen met Typescript, wat zorgt voor meer typeveiligheid, zodat bugs vermeden kunnen worden.
 - **Toekomstbestendigheid:** Next.js wordt voortdurend geüpdate en ondersteund door Vercel, wat dus betekent dat Next.js blijft evolueren met de nieuwe technologieën.
 - **Routing:** Next.js maakt gebruik van file-based routing. Dit stelt ons in staat om aan de hand van de mappenstructuur eenvoudig pagina's en navigatie op te zetten, zonder complexe configuratie.
- **CSS**
 - **Beschrijving:** CSS is de standaardtaal die wordt gebruikt voor de opmaak, het visuele ontwerp en de lay-out van webpagina's.
 - **Argumenten:**
 - **Volledige Controle:** met standaard CSS heb je de absolute vrijheid en controle over elk ontwerpaspect tot op de pixel nauwkeurig, zonder beperkt te worden door de voor gedefinieerde regels van een framework.
 - **Geen Extra Afhankelijkheden:** het gebruik van pure CSS vereist geen externe bibliotheken, installaties of complexe build-tools. Het werkt direct en native in elke webbrowser, wat zorgt voor een eenvoudigere projectopzet.

Backend technologieën:

- **Express**
 - **Beschrijving:** Express is een minimalistisch framework voor Node.js dat in dit project specifiek wordt ingezet als REST API. Terwijl de frontend wordt afgehandeld door Next.js, fungeert Express als de pure backend laag die de bedrijfslogica en dataverwerking verzorgt.
 - **Argumenten:**
 - **Focus op API:** Express is uitstekend geschikt voor het bouwen van lichtgewicht JSON API's.
 - **Middleware Ondersteuning:** het maakt gebruik van middleware, waardoor we eenvoudig functionaliteiten zoals authenticatie, logging en foutafhandeling modulair kunnen toevoegen aan de API-endpoints.
 - **Typescript integratie:** Express werkt naadloos samen met Typescript, wat zorgt voor meer typeveiligheid, zodat bugs vermeden kunnen worden.
- **PostgreSQL**
 - **Beschrijving:** PostgreSQL is een krachtige open source object relationeel databasesysteem. Het staat bekend om zijn betrouwbaarheid, robuustheid en prestaties bij het verwerken van complexe data.
 - **Argumenten:**
 - **Relationele Integriteit:** het is uitstekend geschikt voor gestructureerde data zoals gebruiker, bestellingen en producten en garandeert dat relaties tussen deze data consistent blijven.
 - **Schaalbaarheid:** PostgreSQL kan grote hoeveelheid data en gelijktijdige gebruikers aan.
- **Prisma:**
 - **Beschrijving:** Prisma is een moderne ORM die communicatie tussen backend code en de database vereenvoudigt.
 - **Argumenten:**
 - **Typeveiligheid:** Prisma genereert automatisch types op basis van ons database schema, wat perfect samenwerkt met Typescript om fouten tijdens ontwikkeling te voorkomen.
 - **Migraties:** Prisma maakt het eenvoudig om wijzigingen die we lokaal maken veilig te pushen naar de productiedatabase.
- **Swagger**
 - **Beschrijving:** Swagger is een tool die ons in staat stelt om onze REST-API te documenteren en te gebruiken.
 - **Argumenten:**
 - **Interactieve Documentatie:** het genereert automatisch een visuele interface waar ontwikkelaars en testers endpoints direct kunnen uitproberen zonder code te schrijven.
 - **Duidelijkheid voor Frontend:** Het fungeert als het contract tussen backend en frontend. Ontwikkelaars zien precies welke data ze moeten sturen en wat ze terugkrijgen.

Hosting en Deployment:

Volgens de vereisten van de stageopdracht zal de applicatie in eerste instantie volledig functioneren op een lokale omgeving (Localhost). Het onderstaande cloud landschap is uitgewerkt als voorkeursarchitectuur indien er besloten wordt om over te gaan tot publicatie.

- **Neon voor de database**
 - **Beschrijving:** Neon is een serverless PostgreSQL platform gebouwd voor de cloud. Het scheidt opslag van rekenkracht om moderne ontwikkelingsflows mogelijk te maken.
 - **Argumenten:**
 - **Serverless Schaalbaarheid:** Neon schaaft automatisch mee met het gebruik, wat kosten bespaart en prestaties garandeert.
 - **Eenvoudige Deployments:** het werkt naadloos samen met Prisma, we kunnen lokale wijzigingen direct naar de cloud pushen via de command line.
- **Koyeb voor de backend**
 - **Beschrijving:** Koyeb is een serverless platform speciaal ontworpen om full-stack applicaties en API's wereldwijd te deployen.
 - **Argumenten:**
 - **Git-based Deployment:** zodra we code pushen naar GitHub, bouwt en deployt Koyeb automatisch de nieuwe versie van de Express-applicatie.
 - **Geen Configuratiezorgen:** Koyeb beheert de infrastructuur, zodat wij ons enkel op de backend-code hoeven te focussen.
- **Vercel voor de frontend**
 - **Beschrijving:** Vercel is een cloudplatform voor frontend frameworks en statische sites, gebouwd door de makers van Next.js.
 - **Argumenten:**
 - **Next.js Optimalisatie:** omdat Vercel de maker is van Next.js, biedt het de best mogelijke prestaties en ondersteuning voor features zoals server-side rendering.
 - **CI/CD Integratie:** elke keer als er een wijziging naar de repository wordt gepusht, wordt er automatisch een nieuwe versie live gezet.

Testing:

De primaire focus ligt op het garanderen van codekwaliteit middels Unit Testing. Uitgebreidere testvormen (Integratie & E2E) zijn opgenomen als nice to have.

- **Jest:**
 - **Beschrijving:** Jest is een populair JavaScript testing framework dat bekend staat om zijn eenvoud en snelheid. In dit project gebruiken we Jest primair voor het valideren van de backend code met Unit Testing.
 - **Argumenten:**
 - **Snelheid en Prestaties:** Jest voert tests parallel uit in aparte processen, wat zorgt voor zeer snelle feedbackloops tijdens het ontwikkelen van de API.
 - **Mocking Capabilities:** het beschikt over krachtige ingebouwde functies om databases en externe services te mocken. Hierdoor

kunnen we de backend logica geïsoleerd testen zonder telkens de echte database te hoeven aanspreken.

- **Zero Config:** Jest werkt out-of-the-box uitstekend samen met Node.js en TypeScript, waardoor we minimale tijd kwijt zijn aan configuratie en ons kunnen focussen op het schrijven van tests.
- **Playwright:**
 - **Beschrijving:** Playwright is een modern framework voor end-to-end testen. Deze tool is geselecteerd om eventueel de volledige gebruikerservaring te simuleren indien de tijd dit toelaat.
 - **Argumenten:**
 - **Cross-Browser Ondersteuning:** Playwright kan tests tegelijkertijd uitvoeren op alle moderne browser-engines, wat garandeert dat de webshop voor elke bezoeker werkt.
 - **Betrouwbaarheid:** in tegenstelling tot oudere tools wacht Playwright automatisch tot elementen op het scherm zichtbaar en interactief zijn voordat hij actie onderneemt.

Versiebeheer:

- **GitHub:**
 - **Beschrijving:** GitHub is een platform dat wordt gebruikt voor softwareontwikkeling en versiebeheer. Het zorgt ervoor dat ontwikkelaars in staat zijn om samen aan projecten te werken, zo kunnen ze code beheren in een georganiseerde omgeving.
 - **Argumenten:**
 - **Veel gebruikt:** GitHub wordt veel gebruikt voor het beheren van grote projecten.
 - **Interface:** GitHub heeft een zeer simpele user interface waarbij je snel en makkelijk het project kan beheren.

e) Impact op de huidige infrastructuur

Voor de realisatie van de BrightSwagShop is er gekozen voor een moderne, gedistribueerde architectuur. Conform de vereisten zal de applicatie in de basis volledig lokaal draaien op de infrastructuur van de ontwikkelaar, zonder dat er externe servers of licenties aangekocht hoeven te worden.

Hoewel de focus ligt op lokale ontwikkeling, is het ontwerp volledig voorbereid op een eventuele transitie naar de Cloud. Indien de optionele deployment wordt uitgevoerd, is de architectuur zo opgezet dat de Next.js frontend geoptimaliseerd is voor hosting op Vercel, de Express REST API voorbereid is voor het serverless platform Koyeb en de dataopslag gefaciliteerd kan worden door Neon.

Wat betreft de kostenanalyse is het project momenteel budgetneutraal. Zowel de lokale ontwikkelomgeving als de benoemde cloud-diensten, de Free Tiers en hobby-abonnementen vereisen geen initiële investeringskosten of operationele uitgaven. Aangezien dit een nieuwbouwproject betreft, zijn er geen bestaande systemen die gewijzigd of verwijderd moeten worden, de gehele infrastructuur wordt van nul opgezet.

f) Analyse van security

Om ervoor te zorgen dat de webapplicatie beveiligd is zullen we een aantal stappen ondernemen om kwetsbaarheden te identificeren, risico's te minimaliseren en de veiligheid te versterken, zodat de gegevens van de gebruikers beschermd blijven.

- **GDPR**
 - **Dataverzameling:** we verzamelen alleen de noodzakelijke gebruikersgegevens die nodig zijn voor het platform.
- **HTTPS**
 - **SSL/TLS certificaten:** in een productie-omgeving zal alle communicatie tussen de client, de server en de database verlopen via HTTPS, gefaciliteerd door hostingproviders die automatisch SSL-certificaten genereren. Tijdens de lokale ontwikkeling simuleren we deze beveiliging, waarbij de architectuur zo is opgezet dat een overstap naar een beveiligde verbinding naadloos mogelijk is.
- **CORS**
 - Omdat de frontend en backend op verschillende domeinen draaien, configureren we strikte CORS-regels in de Express applicatie.
- **JWT**
 - Voor het beveiligen van de sessies maken we gebruik van JSON Web Tokens. Wanneer een gebruiker inlogt, genereert de backend een versleuteld token dat de identiteit van de gebruiker bevestigt.
- **Validatie:**
 - We vertrouwen nooit data die van de gebruiker komt. We zullen alle inkomende JSON-data op de backend strikt valideren voordat we er iets mee doen.
- **SQL Injection Preventie:**
 - Door het gebruik van Prisma ORM worden database query's automatisch geparameteriseerd. Dit maakt de applicatie immuun voor klassieke SQL-injectie aanvallen, omdat invoer niet als uitvoerbare code wordt geïnterpreteerd.
- **Rate Limiting:**
 - Om te voorkomen dat hackers duizenden keren per seconde proberen in te loggen, implementeren we rate limiting op de API.
- **Environment Variables:**
 - Gevoelige informatie zoals database-URL's, API-sleutels en JWT-geheimen worden nooit hardcoded in de broncode opgeslagen. We maken gebruik van .env bestanden die niet worden geüpload naar GitHub, en injecteren deze variabelen veilig via het dashboard van de hostingprovider.
- **Sterk wachtwoordbeleid**
 - **Wachtwoordvereisten:**
 - We dwingen sterke wachtwoorden af om brute force aanvallen te bemoeilijken. Een wachtwoord moet minimaal 12 tekens bevatten en bestaan uit een combinatie van hoofdletters, kleine letters, cijfers en speciale tekens.
 - **Wachtwoord hashing:**

- We gebruiken Bcrypt: Bcrypt voegt automatisch een salt toe en is traag van opzet, waardoor het voor hackers extreem moeilijk is om wachtwoorden te kraken, zelfs als de database lekt.

- Testing

- Tot slot borgen we de veiligheid door middel van tests. Met Unit tests verifiëren we dat foutieve input correct wordt afgewezen zonder gevoelige systeeminfo te lekken. Indien de scope het toelaat, controleren we via API tests of endpoints correct zijn afgeschermd voor onbevoegde rollen en simuleren we via Frontend tests of afgeschermdde pagina's daadwerkelijk onbereikbaar zijn voor niet-ingelogde gebruikers.

Restrisico's (Kan het gehackt worden?)

Hoewel geen enkel systeem volledig waterdicht is, minimaliseren we restrisico's zoals social engineering, supply chain attacks en zero-day exploits door middel van regelmatige security audits en het consequent doorvoeren van updates.

g) Autorisatie rollen

Binnen de BrightSwagShop wordt gewerkt met een strikte scheiding van rechten. Omdat de applicatie dient als testobject met gescheiden omgevingen, is het cruciaal dat beheertaken voor de infrastructuur gescheiden blijven van de operationele trainingstaken.

Hieronder vind je een overzicht van de autorisatie rollen binnen het systeem, samen met de beschrijving van hun mogelijkheden en beperkingen.

Super Admin:

De Super Admin heeft de hoogste rechten en richt zich puur op het faciliteren van de omgeving voor klanten en trainers.

Mogelijkheden:

- **Instantiebeheer:** aanmaken van nieuwe, geïsoleerde omgevingen voor specifieke trainingen of klanten.
- **Gebruikersbeheer:** aanmaken van Super Admin, Admin en Gebruikers accounts en deze koppelen aan de juiste instantie.
- **Productbeheer:** beheren van de globale productcatalogus die overal zichtbaar is.

Beperkingen:

- **Geen operationele invloed:** de Super Admin houdt zich niet bezig met het verloop van een specifieke training of demo.
- **Geen testuitvoer:** Deze rol is niet bedoeld om bestellingen te plaatsen of flows te testen.

Admin:

De Admin is de beheerder van één specifieke instantie. In de praktijk is dit vaak de trainer of demo gever die de applicatie configureert voor gebruik.

Mogelijkheden:

- **Bug Configuratie:** het real-time in- en uitschakelen van specifieke bugs om testscenario's te faciliteren.

- **Data Management:** het resetten van de database om alle fake-bestellingen van de vorige sessie te verwijderen.
- **Gebruikersbeheer:** aanmaken van Gebruiker accounts en deze koppelen aan de juiste instantie.
- **Demo-modus:** een directe toegang tot de webshop om bugs te demonstreren.

Beperkingen:

- **Isolatie:** de Admin heeft geen toegang tot de instellingen of data van andere instanties.
- **Geen infrastructuur:** de Admin kan zelf geen nieuwe instantie aanmaken, hij opereert binnen de omgeving die hij heeft gekregen.

Gebruiker:

De Gebruiker is de persoon die de applicatie gebruikt waarvoor hij bedoeld is, het uitvoeren van testautomatisering.

Mogelijkheden:

- **Instantie Selectie:** kiezen op welke omgeving hij wil inloggen.
- **Winkelproces:** volledige toegang tot de webshop, producten zoeken, winkelmandje beheren en fake-betalingen uitvoeren.
- **Validatie:** inzien van eigen bestelgeschiedenis en profielgegevens om te verifiëren of tests geslaagd zijn.

Beperkingen:

- **Geen configuratie:** de gebruiker kan geen bugs aan- of uitzetten.
- **Privacy:** de Gebruiker ziet enkel zijn eigen bestellingen en niet die van andere personen of testers.

F. Documentatie

Voor de algemene projectdocumentatie en de gebruikershandleiding maken we simpelweg gebruik van Word. Toch hanteren we voor de technische kant van het project een andere, meer directe aanpak, we willen externe documentatie over de code tot een absoluut minimum beperken.

Ontwikkelaar documentatie: voor ontwikkelaars moet de code voor zichzelf spreken. Door heldere naamgeving en een logische opbouw (Clean Code) is het lezen van externe documentatie overbodig. We schrijven dan ook geen uitgebreide functie uitleg in Word. In plaats daarvan plaatsen we uitsluitend beknopte comments in de code om uit te leggen waarom een complexe keuze is gemaakt, niet wat de code precies doet. Alleen de minimale basis, zoals de setup voor de lokale- en productieomgeving, wordt kort in het README-bestand vastgelegd om nieuwe teamleden vlot op weg te helpen.

Gebruikersdocumentatie: deze documentatie is bedoeld voor de gebruikers van de website. Het zal een gedetailleerde uitleg geven van de stappen die nodig zijn voor het uitvoeren van bepaalde acties op de website. De documentatie zal helder en duidelijk worden opgesteld, dit zorgt ervoor dat de gebruikers eenvoudig de informatie terug kunnen vinden waar nodig. Daarnaast zal deze documentatie ook beschikbaar zijn op de website, waardoor de gebruikers op elk moment toegang hebben tot de nodige ondersteuning.

III. Planning

De totale doorlooptijd voor dit project bedraagt 10 weken. Omdat de eerste week volledig gereserveerd is voor de project analyse, blijven er nog 9 effectieve weken over voor de technische realisatie.

Er zal worden gewerkt volgens de Scrum methodiek in sprints van 2 weken. Op basis van de beschikbare tijd vertaalt dit zich naar 4 volledige sprints, gevolgd door een afsluitende 9^{de} week voor documentatie en polish. De onderstaande sprint planning is op deze indeling gebaseerd.

Sprint 1 Week 2-3

Totaal Sprint 1: 26SP

Type	Ticket Omschrijving	Use Case	SP
BE	Datamodel artikelen	Product details bekijken / Producten raadplegen	2
BE	API endpoint lijst artikelen	Zoeken & Filteren / Producten raadplegen / Product details bekijken	3
FE	Publieke landingspagina	Geen.	3
FE	Productoverzicht	Producten raadplegen / Zoeken & Filteren	5
BE	Authenticatie & gebruikersmodel	Inloggen / Registreren	3
BE	Login endpoint	Inloggen	3
BE	Rollen & autorisatie	Inloggen / Accountinstellingen aanpassen (Rollen)	3
BE	Bugs lijst endpoint	Bugs configureren	2
FE	Registratie UI basis	Registreren	2

Sprint 2 – Week 4-5

Totaal Sprint 1: 28SP

Type	Ticket Omschrijving	Use Case	SP
BE	Registratie & validatie	Registreren	5
BE	Instanties aanmaken + BugConfig	Nieuwe instantie aanmaken / Bugs configureren	5

FE	Registratie met instantie-keuze	Registreren / Instantie selecteren	5
BE	Admin bug config endpoint	Bugs configureren	3
FE	Admin overzicht basis	Dashboard bekijken	5
FE	Demo actief indicator	Dashboard bekijken	3
BE	Bug: producten laden niet	Bugs configureren	3

Sprint 3 – Week 6-7

Totaal Sprint 3: 28SP

Type	Ticket Omschrijving	Use Case	SP
BE	API endpoint winkelmandje aanmaken	Winkelmandje beheren	3
BE	API endpoint winkelmandje ophalen	Winkelmandje bekijken	2
FE	Product toevoegen aan winkelwagen knop	Winkelmandje beheren	2
FE	Winkelmand toevoegen (Feedback & Overzicht)	Winkelmandje beheren	3
FE	Winkelwagen overzicht	Winkelmandje bekijken	5
BE	API endpoint fake checkout	Bestelling plaatsen / Betaling simuleren	5
FE	Fake checkout flow	Bestelling plaatsen / Betaling simuleren	3
BE	Bug dubbel product	Bugs configureren	2
BE	Bug prijsberekening	Bugs configureren	3

Sprint 4 – Week 8-9

Totaal Sprint 4: 29SP

Type	Ticket Omschrijving	Use Case	SP
FE	Admin bug toggles	Bugs configureren	3

BE	CRUD admins API	Gebruikers raadplegen / Gebruikers toevoegen / Gebruikers aanpassen / Gebruikers verwijderen	5
FE	Superadmin adminbeheer	Admins raadplegen / Admin toevoegen / Admin aanpassen / Admin verwijderen / Super Admin toevoegen / Super Admin aanpassen / Super Admin verwijderen	8
FE	Super Admin instantie beheer	Instantie raadplegen / Instantie details bekijken / Instantie toevoegen / Instantie aanpassen / Instantie verwijderen	5
BE	Instantie beheren/reset/verwijderen	Instantie raadplegen / Instantie details bekijken / Instantie aanpassen / Instantie verwijderen / Test data resetten	8

Sprint 5 – Week 10

Totaal Sprint 5: 15SP

Type	Ticket Omschrijving	Use Case	SP
BE	API documentatie (Swagger)	Geen.	5
FE	UI polish	Geen.	3
FE	bugfixes	Geen.	3
BE	Security check	Geen.	2
BE	Code cleanup	Geen.	2

IV. Samenvatting

In opdracht van Brightest wordt de BrightSwagShop ontwikkeld. Dit is een op maat gemaakte fake webshop die dient als gecontroleerd testobject voor trainingen, commerciële demo's en rekrutering. De huidige situatie, waarbij geschikte testsoftware ontbreekt of onvoorspelbaar is, wordt opgelost door een flexibel platform waarin bugs via een dashboard in- en uitgeschakeld kunnen worden. Dankzij een strikte scheiding van rechten kunnen meerdere gebruikersgroepen gelijktijdig en geïsoleerd werken zonder elkaars testdata te beïnvloeden.

Technisch wordt er gekozen voor een moderne, gedistribueerde architectuur met een frontend in Next.js en een backend in Express en PostgreSQL. Hoewel de focus ligt op een lokale implementatie, is het systeem volledig voorbereid op een eventuele cloud deployment. Deze opzet, ondersteund geautomatiseerde tests en CI/CD-pijplijnen, dit zorgt niet alleen voor tijdbesparing bij trainers, maar biedt ook een stabiele en schaalbare omgeving om de kracht van testautomatisering effectief aan te tonen.

Professionele Bachelor Toegepaste Informatica

Testplan

Onderdeel van de stage
ondersteund door de

AP Hogeschool

en uitgevoerd op en begeleid door het bedrijf

Brightest

Milan Mareels

Specialisatie It & Software

Begeleider: David Verhulst
Mentor: Cat Ooms

Academiejaar 2025-2026
2^{de} semester

A. Inleiding

Dit testplan is primair opgesteld voor de stakeholders van het BrightSwagShop project, waaronder de begeleidende docenten, de opdrachtgever vanuit Brightest en het ontwikkelteam.

Voor dit specifieke project is een doordacht testplan cruciaal. De BrightSwagShop is geen standaard e-commerce applicatie, maar een educatief platform dat doelbewust geconfigureerd wordt om voorspelbare fouten "bugs" te vertonen. Dit stelt gebruikers in staat om hun testautomatisatiescripts te valideren. Het is essentieel te garanderen dat geactiveerde bugs uitsluitend effect hebben binnen de geïsoleerde testomgeving van de trainer of klant. Omdat de architectuur leunt op multi-tenancy, zoals uitgewerkt in de Blueprint BrightSwagShop, brengt een fout in data isolatie directe risico's mee voor andere actieve sessies. Dit testplan waarborgt de strikte datascheiding tussen de Super Admin, de Admin en de eindgebruiker, zodat data elkaar nooit overlapt.

In de volgende hoofdstukken krijg je een overzicht van de scope, de belangrijkste projectrisico's en de testomgeving. Daarnaast leg ik de testaanpak voor de backend en frontend uit. Een praktische teststrategie, met focus op multi-tenancy en de bug-toggles. Dit is de basis voor een succesvolle oplevering.

B. Projectbeschrijving en scope

In opdracht van Brightest wordt de BrightSwagShop ontwikkeld. Dit is een "fake" webshop die dient als testobject voor trainingen, demo's en rekrutering. Het platform verhelpt het gebrek aan voorspelbare testsoftware door programmeerfouten "bugs" via een dashboard in- en uit te schakelen. Een kernonderdeel is de strikte scheiding van rechten en data via een multi-tenancy architectuur. Hierdoor kunnen gebruikersgroepen gelijktijdig en volledig geïsoleerd werken, zonder elkaars testdata te beïnvloeden.

De functionaliteiten zijn verdeeld over drie rollen. De Super Admin beheert de infrastructuur en maakt centraal nieuwe instanties en beheerdersaccounts aan. De Admin trainer of recruiter kan via een beheerpaneel de testdata van zijn instantie resetten en webshop bugs configureren. Tot slot is er de Gebruiker, die inlogt op de omgeving en de reguliere webshop flows doorloopt.

Technisch bestaat de applicatie uit een frontend in Next.js en een backend in Express en PostgreSQL. De focus ligt op een lokale implementatie, hoewel het systeem cloud ready is. Echte e-commerce processen, zoals betalingstransacties, vallen buiten de scope en worden gesimuleerd. Het einddoel is een stabiele omgeving faciliteren die de kracht van testautomatisering aantoont.

C. Belanghebbenden

Naam	Bijdrage/Verantwoordelijkheid
Milan Mareels	Ontwikkelaar, opstellen en uitvoeren van testplan
Cat Ooms	Projectmanagement + Nalezen testplan
Brahim El Mossaoui	Testen + Development

D. Risicoanalyse

Het risico is de kans dat de gebeurtenis optreedt indien er niets gedaan wordt:

Risico	Betekenis
0	Kan nooit optreden
1	Héél erg kleine kans
2	Matige kans
3	Grote kans
4	Erg grote kans
5	Gedurende de levensduur van de applicatie zal de gebeurtenis quasi zeker minstens éénmaal plaatsvinden

De impact is een inschatting van de ernst van de gevolgen op de normale werking van de applicatie indien het risico niet afgedekt wordt:

Risico	Betekenis
0	Geen effect, ook niet op lange termijn, noch voor de eindgebruiker als de eigenaar van de applicatie
1	De impact voor de eindgebruiker is minimaal of van zeer korte duur óf de impact situeert zich achter de schermen en heeft bijvoorbeeld op langere termijn een invloed op de herbruikbaarheid of de onderhoudbaarheid van de broncode
2	Een aantal functionaliteiten zijn tijdelijk niet beschikbaar voor de eindgebruiker
3	Een belangrijk deel van de functionaliteiten van de applicatie is (tijdelijk) niet beschikbaar
4	De meeste functionaliteit van de applicatie is verloren maar er is een mogelijkheid om een aantal problemen (tijdelijk) te overbruggen
5	Geen enkele functionaliteit van de applicatie is beschikbaar.

De bedoeling van deze risicoanalyse is om de activiteiten binnen de teststrategie te prioriteren. Omdat het in de praktijk vaak onmogelijk is om elke randvoorwaarde en onwaarschijnlijke flow te testen, helpt deze analyse om te bepalen welke functionaliteiten we als eerste en meest grondig moeten afdekken. De identificatie van deze risico's is tot stand gekomen door middel van een architectuuranalyse van de Blueprint en door overleg met de domeinexperten van het stagebedrijf Brightest. De prioriteit per risico is berekend aan de hand van de voorgeschreven formule: $\text{rnd}(\frac{1}{5} \times \text{risico} \times \text{impact})$. Hieronder zijn

de 10 meest kritieke, projectspecifieke risico's voor de BrightSwagShop geïdentificeerd en overzichtelijk gesorteerd van hoge naar lage prioriteit.

1. Data overslag tussen gescheiden instanties

Omschrijving	Risico	Impact	Prior.
De multi-tenancy architectuur faalt. Gebruikers van instantie A zien testdata of configuraties van instantie B in de database.	3	5	3
Actie	Actietype		
We schrijven strikte unit tests en manuele tests om de validatie van het Instanceld bij elke afzonderlijke database query te verifiëren. Hiermee garanderen we dat de datascheiding altijd behouden blijft.	Inperken		

2. Bug configuraties lekken naar ongeautoriseerde omgevingen

Omschrijving	Risico	Impact	Prior.
Een specifieke bug zoals een crashende checkout die geactiveerd wordt door Admin A treedt globaal op, in plaats van uitsluitend in diens eigen omgeving.	3	5	3
Actie	Actietype		
Er worden geïsoleerde manuele tests gedaan voor de state management van de bugs. We testen of de instellingen enkel effect hebben op gebruikers met hetzelfde Instanceld.	Inperken		

3. SQL-Injectie door testscripts

Omschrijving	Risico	Impact	Prior.
Omdat het platform dient voor testautomatisering, kunnen cursisten malafide SQL-payloads sturen via invoervelden zoals de login. Zonder goede beveiliging kan dit de PostgreSQL database corrumperen of de multi-tenancy doorbreken.	3	5	3
Actie	Actietype		
We testen manueel of SQL-injectie onmogelijk is, en we maken uitsluitend gebruik van een ORM in de Express backend om directe database-manipulatie onmogelijk te maken.	Vermijden		

4. API overbelasting door scripts cursisten

Omschrijving	Risico	Impact	Prior.
Foutief geschreven geautomatiseerde testscripts van cursisten zoals oneindige loops overbelasten de Express backend, wat leidt tot downtime.	4	4	3
Actie	Actietype		
Implementatie en validatie van server side rate limiting op alle backend API endpoints om overbelasting te voorkomen. We testen dit door zelf massale requests te sturen.	Inperken		

5. Frontend caching toont oude bug-status

Omschrijving	Risico	Impact	Prior.
De Next.js frontend cachet de actieve status van een bug. Na uitschakeling toont de UI de programmeerfout lokaal nog steeds.	4	3	2
Actie	Actietype		
Cache-invalidatie strategieën toepassen. We testen manueel in de browser of state-veranderingen vanuit het admin dashboard direct correct doorsijpelen naar de webshop.	Inperken		

6. Ongeautoriseerde toegang tot beheerpaneel

Omschrijving	Risico	Impact	Prior.
Een cursist vindt de URL van het admin dashboard en past zelf de configuratie van bugs aan, wat evaluaties verstoort.	2	4	2
Actie	Actietype		
Beveiligingstesten uitvoeren op de toegang. We valideren role-based access control en JWT tokens om te testen of niet-admins consistent een 403 Forbidden error krijgen.	Inperken		

7. Bug-toggle genegeerd door backend logica

Omschrijving	Risico	Impact	Prior.
De Admin schakelt een fout in, maar de backend implementeert de status niet correct in de daadwerkelijke testflow.	3	3	2

Omschrijving	Risico	Impact	Prior.
Actie	Actietype		

Manuele testen uitvoeren die de volledige logica valideren: van de druk op de knop in het dashboard tot de verwachte foutmelding in de frontend van de webshop.

Inperken

8. Data Reset botst met actieve testscripts

Omschrijving	Risico	Impact	Prior.
--------------	--------	--------	--------

De Admin voert via het dashboard een "Data Reset" uit, exact op het moment dat cursisten geautomatiseerde testscripts draaien. Testdata verdwijnt onverwachts tijdens de uitvoering.

3

3

2

Actie	Actietype
-------	-----------

We testen de afhandeling van gelijktijdige verzoeken door de API actief te bevragen tijdens een reset-actie. We gaan de testdata pas resetten als alle bugs op false staan en alles actief draait.

Inperken

9. Swagger API documentatie loopt asynchroon

Omschrijving	Risico	Impact	Prior.
--------------	--------	--------	--------

De API-documentatie loopt achter op de backend code. Developers baseren hun scripts op verkeerde Swagger-data en krijgen onterechte errors.

5

1

1

Actie	Actietype
-------	-----------

Hoewel dit frustrerend is voor de gebruiker, breekt het de applicatie zelf niet. We vangen dit deels op door Swagger generatie te automatiseren vanuit de code annotations.

Aanvaarden

10. Databank connectie-pool uitgeput

Omschrijving	Risico	Impact	Prior.
--------------	--------	--------	--------

Doordat vele cursisten tegelijkertijd testscenario's draaien, raakt de PostgreSQL connectie pool uitgeput. Dit leidt tot time-outs.

2

3

1

Actie	Actietype
-------	-----------

Omschrijving	Risico	Impact	Prior.
Door het gebruik van een serverless PostgreSQL database wordt de belasting en het beheer van actieve connecties deels automatisch opgevangen.		Inperken	

E. Teststrategie

Binnen dit project wordt een iteratieve aanpak gehanteerd waarbij kwaliteitscontrole een continu proces vormt. Omdat we dit traject als een team van twee ontwikkelaars uitvoeren, verdelen we de verantwoordelijkheden voor zowel de ontwikkeling als het testwerk. We hanteren hierbij consequent het vierogenprincipe, functionaliteiten die door het ene teamlid zijn geprogrammeerd, worden door het andere teamlid gereviewd en getest voordat deze definitief worden samengevoegd.

Om de werklast te optimaliseren, is er sterk ingezet op automatisering binnen de ontwikkelpijplijn (CI/CD). De fundering van de applicatie wordt gevalideerd via unit testen die automatisch draaien in een GitHub Action bij elke nieuwe Pull Request. Dit garandeert dat de nieuwe code de bestaande logica niet breekt.

Voor de overige lagen van de applicatie leunt de strategie op gerichte, manuele validatie door de backend manueel te testen aan de hand van Swagger om te verifiëren of de correcte HTTP-statuscodes en JSON-structuren worden geretourneerd. De frontend wordt eveneens onderling manueel gevalideerd. Hierbij wordt de gebruikersinterface doorlopen en gecontroleerd. Indien de projectvoortgang en de beschikbare tijd het echter toelaten, is er een ambitie om ook deze testlagen verder te automatiseren. Het doel is dan om de manuele API-calls te vervangen door geautomatiseerde Jest-integratietesten en de frontend te voorzien van E2E testen via Playwright, waarna we beide toevoegen aan onze GitHub Actions pipeline.

Hieronder volgt een overzicht van de testtypes, of deze gepland zijn binnen de scope, en de bijhorende meetbare criteria.

Testtype	Gepland?	Bereik en criteria
Unit testen	Ja	Focus op de backend logica. Deze draaien geautomatiseerd via een GitHub Action bij elke PR. Criterium: Het streefdoel is dat 100% van de pipeline testen succesvol slagen voordat een PR gemerged wordt, maar we stellen hierop bewust geen harde technische blokkade in om flexibel te blijven. Daarnaast streven we naar een test coverage van 80%.
Integratie testen	Ja	Focus op de communicatie tussen de backend en de database. Criterium: Geslaagd wanneer de API-aanroepen de PostgreSQL database correct updaten zonder connectiefouten.
Systeem testen	Ja	Manuele E2E validatie van de frontend. Criterium: Geslaagd als een eindgebruiker succesvol de flow van inloggen tot de checkout-simulatie via de gebruikersinterface kan doorlopen, en bugs correct reflecteren.
Sanity testen	Nee	Formele sanity testen worden niet afzonderlijk gedocumenteerd, omdat de snelle checks al gedekt zijn door de geautomatiseerde GitHub Actions en de cross checks tussen de twee ontwikkelaars.
Interface testen	Ja	Validatie van de REST API endpoints. Dit gebeurt manueel via de Swagger UI. Criterium: Geslaagd als de endpoints de verwachte data,

Testtype	Gepland?	Bereik en criteria
		foutmeldingen en HTTP-statuscodes (zoals 200, 400, 403) retourneren.
Regressie testen	Ja	Wordt continu en geautomatiseerd uitgevoerd. Criterium: De GitHub Action pipeline garandeert dat eerder geschreven unit tests blijven slagen bij het toevoegen van nieuwe codewijzigingen.
Beta- of acceptatietesten	Nee	Gezien de aard van dit interne stageproject is er geen formele externe bètafase, de finale oplevering en review door de begeleiders van Brightest fungeert als eindacceptatie.
Performantie testen	Nee	Formele load- of stresstesten vallen buiten de primaire scope van de MVP, omdat de focus ligt op de correcte logische werking van de multi-tenancy en de bugs.
Security testen	Ja	Focus op data isolatie. Criterium: Manuele testen in Swagger moeten aantonen dat ongeautoriseerde verzoeken zonder de juiste JWT-token resulteren in een 403 Forbidden.
Cross-browser testen	Nee	De applicatie dient als intern platform. Visuele perfectie over oude of zeldzame browsers is out of scope, Google Chrome is de vaste baseline voor alle gezamenlijke UI-testen.
Usability testen	Nee	Het optimaliseren van de UX voor reguliere eindgebruikers is niet het doel. De webshop is doelbewust gebouwd om fouten “bugs” te bevatten voor testdoeleinden, en niet om perfect te converteren.

F. Testomgeving

Hieronder beschrijf ik de specifieke hardware, software en cloud service die door ons team van twee ontwikkelaars wordt ingezet om de teststrategie van de BrightSwagShop uit te voeren. Alle testen vinden ofwel lokaal plaats op de ontwikkelmachines, ofwel geautomatiseerd in de cloud via onze CI/CD-pijplijn.

Naam	Versie	Verdelers	Omschrijving	Ref
Jest	30.2	OpenJS Foundation	JavaScript test-framework gebruikt voor het schrijven en uitvoeren van alle backend unit testen.	[1]
Playwright	1.51	Microsoft	Framework gereserveerd voor toekomstige uitbreiding van geautomatiseerde E2E frontend testen.	[2]
Node.js	24.14.0 (LTS)	OpenJS Foundation	De runtime environment waarop zowel de applicatie als de test-scripts lokaal en in de cloud draaien.	[3]

Naam	Versie	Verdelers	Omschrijving	Ref
Express	5.0.6	OpenJS Foundation	Het backend framework waarop de API endpoints draaien die we via Swagger testen.	[4]
Next.js	16.1.6	Vercel	Het frontend framework waarvan we de UI en user flows manueel valideren.	[5]
Swagger UI	6.0.4	SmartBear Software	De interface gegenereerd vanuit onze Express backend om alle REST API-endpoints manueel te testen.	[6]
Google Chrome	145.0.7632.117	Google	Vaste browser voor het uitvoeren van alle manuele UI-testen door de peer reviewer.	[7]
GitHub Actions	N/A	GitHub (Microsoft)	De CI/CD-service verantwoordelijk voor het automatisch triggeren van de testen bij elke PR.	[8]
Prisma ORM	7.3.0	Prisma Data	ORM tool gebruikt voor de veilige database interactie.	[9]
PostgreSQL	18.0	PostgreSQL Global Development Group	De relationele database waarin alle data via Prisma wordt opgeslagen.	[10]

Hardware & Infrastructuur Het manuele testwerk UI en API en de ontwikkeling van de unit testen wordt uitgevoerd op de persoonlijke laptops van de twee ontwikkelaars (Windows, Multi-core CPU, 16/32GB RAM). De geautomatiseerde testpijplijn draait op de cloud infrastructuur van GitHub hosted runners. Deze virtuele machines draaien op een ubuntu-latest image.

G. Testresultaten

Visuele bewijs van de uitgevoerde testen, waaronder de rapportages van Playwright en Jest, is opgenomen in de bijlagen achteraan dit document (zie Bijlage).

Professionele Bachelor Toegepaste Informatica

Test reflectieverslag

Onderdeel van de stage
ondersteund door de

AP Hogeschool

en uitgevoerd op en begeleid door het bedrijf

Brightest

Milan Mareels

Specialisatie IT & Software

Begeleider: David Verhulst
Mentor: Cat Ooms

Academiejaar 2025-2026
2^{de} semester

H. Inleiding

Tijdens mijn stage bij Brightest heb ik, samen met een medestudent, de BrightSwagShop ontwikkeld. Dit is een educatieve webshop met een ingebouwd bug toggle systeem. Hiermee kunnen trainers en recruiters heel gericht fouten in de applicatie aanzetten, wat het een ideale omgeving maakt voor testautomatisering oefeningen of rekruteringen. Omdat we werkten met een complexe multi-tenancy architectuur om alle data goed gescheiden te houden, was een sterke kwaliteitscontrole echt van sterk belang. Binnen dit project hield ik me niet alleen bezig met de ontwikkeling, maar nam ik ook de rol op van het opstellen en uitvoeren van het testplan, waarbij we altijd volgens het vierogenprincipe werkten.

Met het testplan wilde ik vooral aantonen dat de applicatie betrouwbaar en robuust genoeg is voor intern gebruik. Mijn absolute focus lag hierbij op twee zaken: checken of de datascheiding tussen de verschillende instanties waterdicht was, en controleren of de bugs via de toggles correct werden geactiveerd. Voor de scope heb ik me primair gericht op geautomatiseerde unit tests voor de backend, gecombineerd met manuele validatie van de API via Swagger en het testen van de user interface. Niet functionele zaken zoals usability of cross-browser compatibiliteit hebben we bewust buiten scope gehouden. Het platform dient immers als een technische testomgeving en niet als een echte commerciële webshop.

In dit reflectieverslag kijk ik terug op het hele proces. Ik sta stil bij de keuzes die ik in de teststrategie heb gemaakt, de technische obstakels waar ik tegenaan liep tijdens de uitvoering, en natuurlijk hoe ik hierin persoonlijk ben gegroeid als startend Test Automation Engineer.

I. Reflectie

Aanpak en keuzes

Bij het maken van het BrightSwagShop project stonden we voor een belangrijke strategische keuze met betrekking tot onze testaanpak. Omdat we een applicatie bouwden met een complexe multi-tenancy architectuur, waarbij het isoleren van testdata en bug configuraties cruciaal was, besloten we de traditionele testpiramide strikt te volgen. We kozen voor een solide basis van geautomatiseerde backend unit tests via Jest, gecombineerd met een sterke CI/CD-pijplijn in GitHub Actions. De frontend en API-integraties besloten we manueel te valideren via de UI en Swagger.

Een mogelijk alternatief was om direct te starten met E2E testautomatisering via een tool als Playwright. Hoewel dit aantrekkelijk leek voor mijn specialisatie in Test Automation, kozen we bewust om dit in de MVP-fase af te bakenen als een nice to have en niet als een harde eis. In de vroege fases van de ontwikkeling veranderen UI-elementen immers nog zo vaak dat E2E testen snel onstabiel worden en enorm veel onderhoud vroegde. Onze keuze om eerst de API en database via Jest waterdicht te maken, pakte dan ook ontzettend goed uit. Het dwong ons om de business logica los te koppelen van de frontend, wat de algehele codekwaliteit verhoogde. Wat minder goed uitpakte, was de inschatting van de benodigde tijd voor het manuele testwerk. Naarmate het aantal endpoints en functionaliteiten groeide, merkten we dat de manuele validatie via Swagger steeds repetitiever en tijdrovender werd, wat de efficiënte flow van onze Kanban werkwijze soms vertraagde.

Uitvoering en problemen

Tijdens de uitvoering van het testplan botsten we op verschillende technische en methodologische obstakels. Omdat we besloten hadden om de integratietesten manueel via de Swagger UI uit te voeren, moesten we er in de eerste plaats voor zorgen dat we de hoofdapplicatie niet zouden vervuilen met testdata. Dit pakte we effectief aan door een specifieke pre-productie database op te zetten. Hierdoor

konden we onze manuele API-calls veilig uitvoeren zonder risico voor de productieomgeving. Het échte knelpunt van deze manuele aanpak bleek de schaalbaarheid. Naarmate de ontwikkeling vorderde en het aantal API-endpoints bleef groeien, kostte het simpelweg steeds meer tijd om al deze routes één voor één manueel via Swagger te testen. Dit intensieve werk begon op termijn de snelheid van onze Kanban tickets af te remmen. Wat betreft de geautomatiseerde CI/CD pijplijn in GitHub Actions, focusten we ons puur op unit testen en de resulterende regressietesten. Hier was de initiële moeilijkheid om de Prisma ORM database aanroepen correct te mocken met Jest, aangezien we deze testen in de pijplijn in volledige isolatie moesten laten slagen zonder een fysieke database te raken.

Een andere grote moeilijkheid was de uitvoering van de manuele systeemtesten voor de bug-toggles in de frontend. Het testen hiervan bleek in de praktijk uiterst repetitief: ik moest inloggen als Admin, een specifieke bug in het dashboard activeren, uitloggen, inloggen als reguliere User, de webshop flow doorlopen om te verifiëren of de bug daadwerkelijk optrad, en vervolgens het hele proces herhalen om de bug weer uit te zetten. Dit liep anders dan verwacht, aangezien de belasting van deze context-switches hoog lag en de tijdsinvestering te groot werd. Om te voorkomen dat dit ons ontwikkelproces volledig vertraagde, kozen we er in de praktijk voor om deze intensieve flows niet meer bij elke codewijziging te testen, maar slechts nog steekproefsgewijs of bij grotere updates. Hoewel we deze manuele validatie dus wel bleven uitvoeren, toonde de noodzaak om de testfrequentie te verlagen voor mij direct de absolute meerwaarde aan van toekomstige E2E automatisering.

Kwaliteit en effectiviteit

Terugkijkend op de kwaliteit van het testplan, ben ik van mening dat het document een zeer volledige en bruikbare houvast bood, met name dankzij de doorgedreven risicoanalyse. Omdat we expliciet het falen van de multi-tenancy als grootste risico hadden aangeduid, lag onze testfocus daar het hoogst. Dit is buitengewoon effectief gebleken: door gerichte integratietesten uit te voeren die verifieerden of een Admin via een API-call data van een ander instantie kon opvragen, ontdekten we daadwerkelijk autorisatiefouten in de Express backend voordat deze in de main branch terechtkwamen. De testen fungeerden als een onmisbaar vangnet.

Wat echter beter had gekund, is de test coverage op de integratielaag tussen de Next.js frontend en de Express backend. Hoewel beide componenten afzonderlijk succesvol werden getest, merkten we dat integratiefouten, zoals onverwachte validatie, soms pas laat in de manuele testfase werden opgemerkt. Als het testplan iteratiever was aangepast tijdens het project, hadden we hier eerder geautomatiseerde API-testen voor de frontend calls kunnen toevoegen.

Persoonlijk leerproces

Dit project heeft mijn visie op software testing en kwaliteit fundamenteel verbeterd. Technisch heb ik enorme stappen gezet in het configureren van CI/CD pijplijn via GitHub Actions en heb ik geleerd hoe cruciaal het is om een pijplijn zo in te stellen dat falende testen een Pull Request blokkeren. Hierdoor werd het vierogenprincipe niet zomaar een afspraak, maar een technisch afgedwongen regel. Daarnaast heb ik geleerd hoe ik efficiënt de Prisma ORM en relationele databases heb moeten mocken binnen een testomgeving.

Methodologisch was mijn belangrijkste inzicht dat Test Automation voor 80% draait om applicatie-architectuur en testability, en slechts voor 20% om het schrijven van de daadwerkelijke testscripts. Als code niet modulair of geïsoleerd is geschreven, is het nagenoeg onmogelijk om deze geautomatiseerd te testen. Door dit project heb ik de vaardigheid ontwikkeld om al tijdens de analysefase na te denken over hoe een functionaliteit getest gaat worden.

Als ik een vergelijkbaar project opnieuw zou beginnen, zou ik mijn tijd net iets anders indelen. In plaats van uitsluitend te vertrouwen op manuele testen voor de UI in de MVP-fase, zou ik vanaf de start van de ontwikkeling een Playwright-framework opzetten dat uitsluitend de absolute flows valideert. Dit had ons tientallen uren aan repetitief manueel testwerk bespaard tijdens het valideren van nieuwe pull requests. Deze inzichten omtrent de balans tussen unittesten, manuele validatie en E2E automatisering neem ik met veel vertrouwen mee naar mijn toekomstige carrière.

J. Conclusie

In de conclusie van dit project kan ik met trots stellen dat het opgestelde testplan een onmisbare leidraad is geweest voor de ontwikkeling van de BrightSwagShop. Door direct een goede risicoanalyse uit te voeren met een absolute focus op de multi-tenancy architectuur, hebben we de meest kritieke fouten rondom data isolatie succesvol kunnen opvangen. De keuze voor de geautomatiseerde unit testen via Jest en een CI/CD pijplijn in GitHub Actions vormde hierbij een ijzersterke technische basis.

Tegelijkertijd heeft de uitvoering van dit testplan me geconfronteerd met de harde grenzen van manuele validatie. Het repetitieve werk van de manuele testen van API's via Swagger en bug toggels via de UI werd op termijn een tijdrovend probleem in onze Kanban flow. Dit vormde mijn belangrijkste leermoment van deze hele stage: testautomatisering begint niet pas achteraf, maar al in de architectuurfase. Code moet simpelweg testbaar geschreven worden.

Voor toekomstige projecten neem ik de absolute overtuiging mee dat een vroege implementatie van test automation essentieel is om repetitief manueel werk te elimineren.

K. Testresultaten

Visuele bewijs van de uitgevoerde testen, waaronder de rapportages van Playwright en Jest, is opgenomen in de bijlagen achteraan dit document (zie Bijlage).

IT@AP

Toegepaste informatica

Technische documentatie

Onderdeel van de stage
ondersteund door de

AP Hogeschool

en uitgevoerd op en begeleid door het bedrijf

Brightest

Milan Mareels

Specialisatie IT & Software

Begeleider: David Verhulst
Mentor: Cat Ooms

Academiejaar 2025-2026
2^{de} semester

L. Samenvatting van de opdracht

In opdracht van Brightest wordt de BrightSwagShop ontwikkeld. Brightest focust zich op softwarekwaliteit en testautomatisering, en heeft intern een sterke behoefte aan gecontroleerde opleidingsomgevingen voor nieuwe test consultants. Bestaande demo applicaties zijn vaak te stabiel, terwijl testen op echte productieomgevingen uiteraard verboden is. De opdracht bestaat uit het bouwen van een fake e-commerce platform dat doelbewust en controleerbaar defecten kan vertonen.

De oplossing, de BrightSwagShop, het bevat een uniek bug toggle dashboard voor trainers. Hiermee kunnen specifieke fouten in de frontend of backend geactiveerd worden. Cursisten moeten vervolgens geautomatiseerde of manuele tests uitvoeren om deze fouten succesvol te detecteren. Om te voorkomen dat verschillende trainingssessies elkaar beïnvloeden, is de applicatie gebouwd op een strikte multi-tenancy architectuur met volledige data isolatie.

M. Impact op de infrastructuur

De applicatie is ontworpen met een moderne, gedistribueerde architectuur die minimale eisen stelt aan de lokale infrastructuur, maar wel specifieke cloud- of servercomponenten vereist voor deployment.

Lokale Ontwikkeling:

De ontwikkeling en initiële testfases draaien lokaal op de werkstations van de ontwikkelaars via Node.js.

Frontend & Backend Hosting:

De applicatie is opgesplitst in een Next.js frontend en een Express backend REST API. Om volledige consistentie tussen de lokale ontwikkelomgeving en de cloud te garanderen, worden beide componenten gecontaineriseerd aan de hand van Dockerfiles. Deze Docker containers worden gehost via het cloudplatform Render. Om de infrastructuurkosten tijdens de ontwikkel- en testfase op nul te houden, wordt er bewust gebruikgemaakt van de Free Tier. De impact hiervan is dat de containers na een periode van inactiviteit in slaapstand gaan, wat resulteert in een tijdelijke vertraging bij het eerste daaropvolgende gebruik.

Database Infrastructuur:

Er wordt gebruikgemaakt van een relationele PostgreSQL database die via de Prisma ORM wordt benaderd. Omdat dit een volledig nieuw project betreft, is het databasemodel volledig van nul ontworpen en opgezet, er hoefde bij de ontwikkeling dus geen rekening gehouden te worden met legacy data. De databases worden gehost op het serverless Postgres platform Neon, waarbij gebruik wordt gemaakt van de Free Tier. Binnen deze infrastructuur is er de strikte vereiste om twee volledig gescheiden database instanties te beheren: een afgeschermd pre-productie database voor veilige manuele API-validatie en een productiedatabase voor de effectieve eindgebruikers.

Schaalbaarheid:

De gekozen cloud infrastructuur is inherent schaalbaar ontworpen. Het serverless platform Neon schaaft automatisch de rekenkracht op basis van de databasebelasting, waardoor er geen handmatige interventie nodig is bij een toename van het aantal gelijktijdige trainingssessies. Voor de applicatiecontainers op Render kan horizontale schaling worden geactiveerd om extra instances van de frontend of backend te draaien indien de responstijden boven een bepaalde drempelwaarde stijgen.

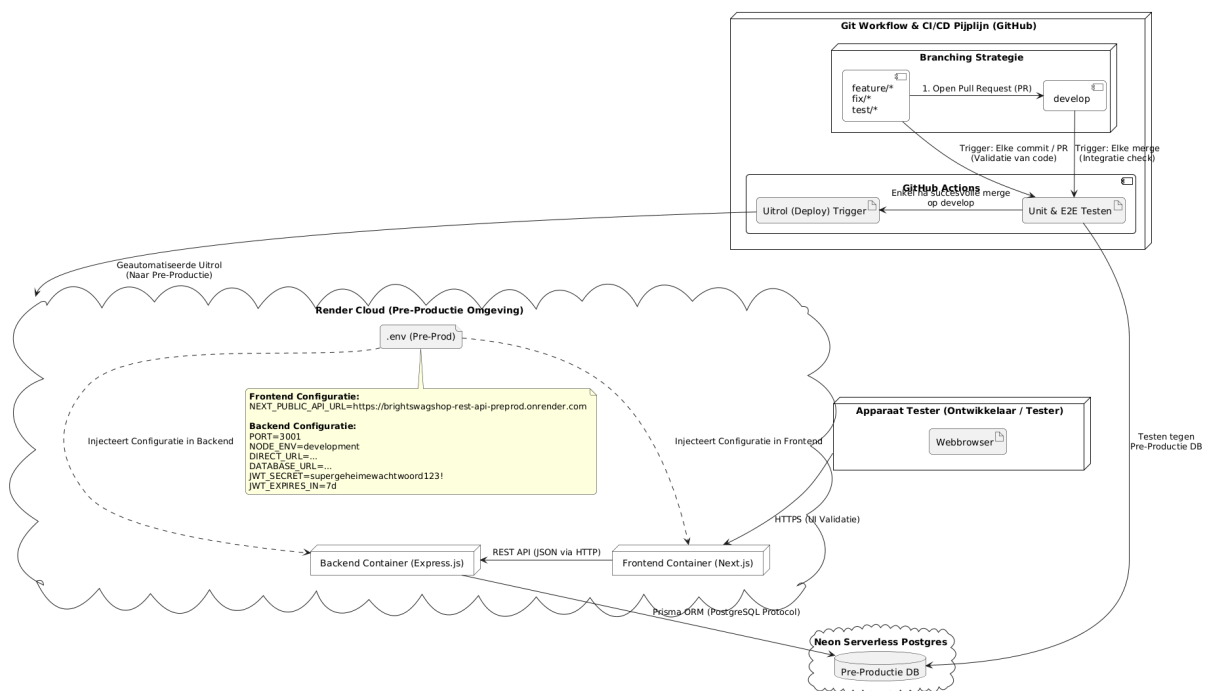
Licenties en Kosten:

Alle gebruikte technologieën (Next.js, Express, Prisma, PostgreSQL) vallen onder permissieve open-source licenties. Voor de hosting wordt momenteel gebruikgemaakt van de Free Tiers van Render en Neon, waardoor de operationele kosten voor Brightest op nul worden gehouden tijdens de ontwikkel- en testfase. Er zijn geen extra licentiekosten verbonden aan de gebruikte softwarebibliotheken of hardwarecomponenten.

CI/CD Infrastructuur:

Voor de testautomatisering en de deployment pijplijn wordt gebruikgemaakt van de cloud infrastructuur van GitHub. Er worden standaard GitHub hosted Ubuntu runners ingezet om de code te bouwen en te valideren. De specifieke stappen van dit proces worden in detail beschreven in het Hoofdstuk Release plan.

Onderstaand deploymentdiagram visualiseert de fysieke opzet van de pre-productieomgeving en de interactie tussen de verschillende cloudcomponenten.



Figuur 29 - Deploymentdiagram Pre-productie

N. Release plan

Het releaseproces van de BrightSwagShop is sterk leunend op Continuous Integration en Continuous Deployment om de stabiliteit van de testomgeving te garanderen. Het release plan verloopt in de volgende vaste stappen.

Installatie-overzicht:

Om de omgeving lokaal of op een nieuwe server op te zetten, moeten de volgende commando's in volgorde worden uitgevoerd:

1. **npm install:** Installatie van alle noodzakelijke dependencies.
2. **npm run migrate:** Initialisatie van het Prisma-schema en database-migraties.

3. **npm run seed:** Het vullen van de database met de initiële catalogus en root-accounts.
4. **npm run dev:** Starten van de applicatie in ontwikkelmodus.

Migratie-instellingen:

Voor de overgang tussen omgevingen wordt gebruikgemaakt van omgevingsvariabelen (.env). Bij de uitrol naar productie moet de DATABASE_URL worden gewijzigd naar de productie instantie in Neon en moet een sterke JWT_SECRET worden gegenereerd. De frontend variabele NEXT_PUBLIC_API_URL dient te verwijzen naar de live productie API op Render.

Versiebeheer & Branching:

Ontwikkeling van nieuwe features en bugfixes gebeurt lokaal op geïsoleerde feature branches. Binnen Git worden twee hoofdbanches gehanteerd: de develop branch fungeert als de pre-productie codebase, en de main branch vertegenwoordigt de stabiele productie codebase.

Continuous Integration:

Bij het aanmaken van een Pull Request naar de develop of main branch, triggert automatisch de GitHub Actions pijplijn. Binnen deze pijplijn worden alle dependencies geïnstalleerd en draaien de geautomatiseerde Jest unit testen en Playwright End To End testen.

Quality Gate:

Code kan pas gemerged worden als de GitHub Actions pijplijn 100% succesvol is afgerond. Falende testen blokkeren de merge technisch op GitHub niveau, wat fungeert als kwaliteitsvangnet.

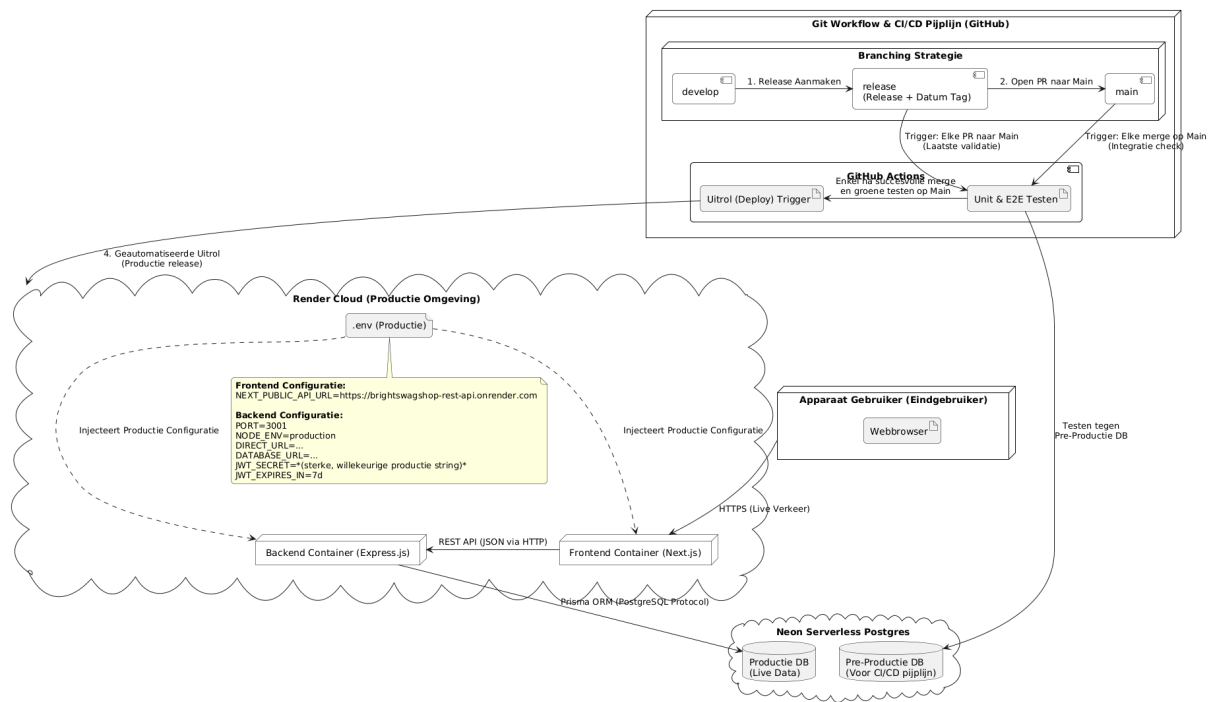
Continuous Deployment & Pre-productie Release:

Na een succesvolle merge naar de develop branch, triggert Render een geautomatiseerde build en deployment. Deze actieve pre-productie omgeving is gekoppeld aan de geïsoleerde Neon pre-productie database. Op deze omgeving worden momenteel de automatische integratietesten en de UI validaties uitgevoerd.

Continuous Deployment & Productie Release:

Hoewel de productie infrastructuur reeds is voorzien, is deze momenteel stand-by. Een daadwerkelijke release naar productie gebeurt pas na volledige goedkeuring op pre-productie, middels een gecontroleerde merge van de develop branch naar release branch en dan naar de main branch.

Onderstaand deploymentdiagram visualiseert de fysieke opzet van de productieomgeving en de interactie tussen de verschillende cloudcomponenten.



Figuur 30 - Deploymentdiagram productie

O. Technisch design

De architectuur van de BrightSwagShop is ontworpen als een gedistribueerde webapplicatie met een strikte ontkoppeling tussen de frontend en de backend.

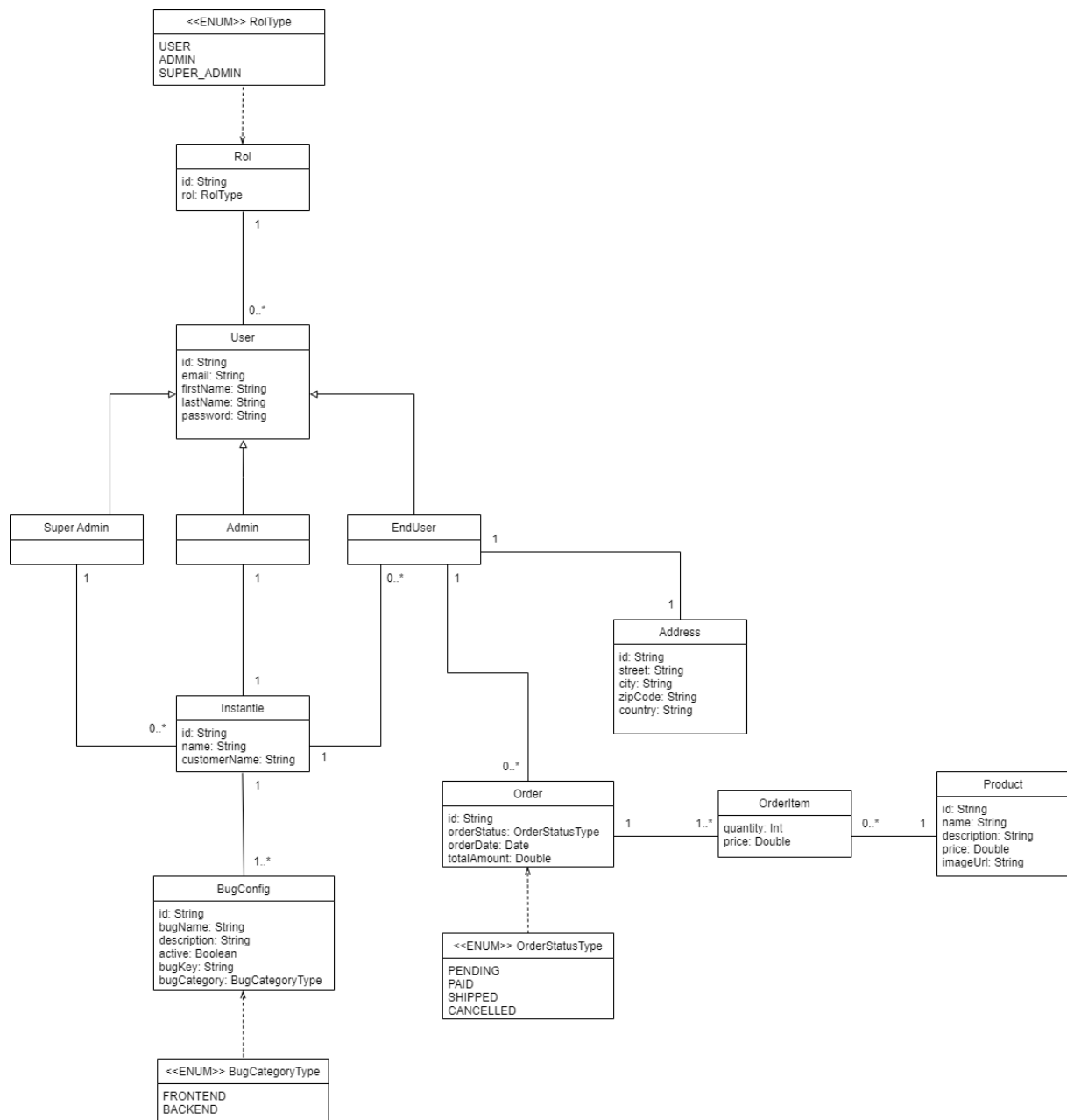
Backend Architectuur:

De REST API, gebouwd in Node.js met Express, volgt een strikte gelaagde architectuur. Deze is opgedeeld in een Controller Layer, een Service Layer en een Data Access Layer.

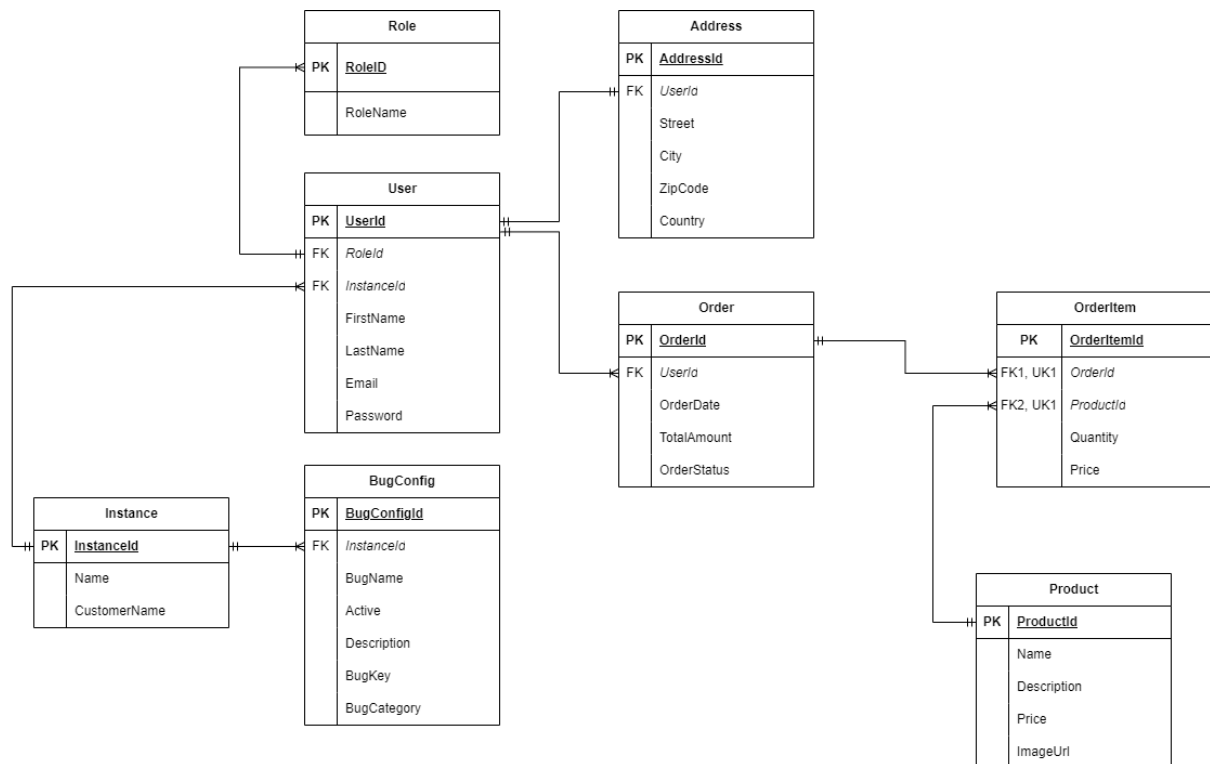
Databasemodellering & Multi-tenancy:

Er wordt gebruik gemaakt van een PostgreSQL database. Een architecturaal concept binnen dit datamodel is de Single Table strategie voor het gebruikersbeheer: alle actoren worden in één centrale User tabel opgeslagen, waarbij een Role kolom als discriminator dient om de rechter te bepalen.

Om de strikte data isolatie te waarborgen, is vrijwel elke kernentiteit via een foreign key hard gekoppeld aan een unieke instantie in de database.



Figuur 31 - Klassen Diagram BrightSwagShop



Figuur 32 - Entity Relationship Diagram BrightSwagShop

P. Externe systeeminterfaces

De BrightSwagShop is ontworpen als een volledig self contained systeem. Om de stabiliteit van de testomgeving te garanderen en externe faalpunten uit te sluiten, zijn er doelbewust geen externe systeeminterfaces of afhankelijkheden met systemen van derden geïmplementeerd.

Interne communicatie:

De communicatie tussen Next.js frontend en de Express backend verloopt via de standaard REST API over HTTP/HTTPS, waarbij alle data uitsluitend in JSON-formaat wordt uitgewisseld.

Mocking van diensten:

Specifieke e-commerce functionaliteiten, zoals betalingsverwerkingen, worden niet afgehandeld via externe API-calls. Deze transacties worden lokaal door de frontend gesimuleerd die de invoer valideert en fictief de betaling uitvoert.

Verantwoording architectuurmodellen:

Omdat de applicatie volledig gesloten is en er geen gegevensstromen zijn naar of van externe systemen, is het uittekenen van een Data Flow Diagram of C4-contextdiagram voor dit specifieke onderdeel niet van toepassing.

Q. Datamigratie

Aangezien de BrightSwagShop een volledig nieuwbouwproject is, is er geen sprake van datamigratie vanuit een legacy systeem of verouderde database. Het databasemodel is volledig van nul opgebouwd. Omdat data niet geïmporteerd wordt uit vervuilde bronnen, is complexe data cleansing overbodig.

Om er echter voor te zorgen dat de applicatie direct bruikbaar is voor trainingen en demo's, wordt de database bij initialisatie voorzien van data via geautomatiseerde database seeding scripts. Dit proces verloopt automatisch en verrijkt de database met de vereiste basisgegevens, waaronder:

- De standaard productcatalogus met Brightest merchandise.
- De benodigde default configuraties voor de bug toggles.
- De initiële root accounts voor de Super Admin rollen.

R. Security en autorisatie rollen

Om de data en de werking van de BrightSwagShop te beschermen, is de security architectuur opgebouwd uit verschillende lagen.

Netwerk & Datatransmissie:

Tijdens lokale ontwikkeling wordt de beveiliging gesimuleerd. Bij de inzet op de productieomgeving verloopt alle communicatie verplicht via HTTPS.

Authenticatie & Sessiebeheer:

De applicatie werkt met een eigen authenticatiesysteem waarbij wachtwoorden versleuteld in de database worden opgeslagen aan de hand van Bcrypt. Om brute force aanvallen af te weren is er een strikt wachtwoordbeleid toegepast. Sessies worden veilig beheerd aan de hand van een JSON Web Token.

Applicatie & Databasebeveiliging:

Om SQL-injectie te voorkomen, maakt de database integratie gebruik van Prisma ORM, de queries worden automatisch geparameteriseerd. Daarnaast implementeert de API een rate-limiter tegen overbelasting en worden alle inkomende JSON-verzoeken op de backend strikt gevalideerd.

Configuratiebeheer:

Gevoelige data zoals JSON Web Tokens en database URL's worden opgeslagen in .env bestanden en worden nooit in de code repository gepusht.

Technische afdwinging van Data-isolatie:

Om de multi-tenancy architectuur veilig te stellen, hanteert de backend een strikte validatie op basis van Instanceld's. Bij elke API-call wordt de Instanceld die is opgeslagen in het JWT-token van de gebruiker vergeleken met de gevraagde data in de database. De middleware op de Express backend zorgt ervoor dat een gebruiker van Instantie A technisch geblokkeerd wordt wanneer deze probeert data op te vragen of te wijzigen die gekoppeld is aan Instantie B. Hierdoor wordt volledige isolatie gegarandeerd, zelfs als gebruikers dezelfde API-endpoints benaderen.

Autorisatie rollen:

De applicatie kent drie strikt gescheiden autorisatie rollen.

1. **Super Admin:** dit is het hoogste niveau. De Super Admin beheert de infrastructuur en is verantwoordelijk voor het creëren van nieuwe geïsoleerde instanties. Deze rol kan overkoepelend nieuwe admins en gebruikers aanmaken en de globale productcatalogus beheren.
2. **Admin:** de Admin heeft uitsluitend beheerrechten binnen zijn eigen toegewezen instantie. Deze rol kan testdata resetten, gebruikers aanmaken en de bug toggles voor zijn sessie configureren.
3. **Gebruiker:** de gebruiker logt in op een specifieke instantie en heeft enkel toegang tot de functionele frontend van de webshop. De gebruiker kan het winkelproces doorlopen en testscripts op af vuren, maar heeft geen toegang tot configuraties en ziet uitsluitend zijn eigen test data.

S. Documentatie

Documentatie in de code:

De technische documentatie voor ontwikkelaars wordt tot een minimum beperkt, waarbij het Clean Code principe centraal staat. In plaats van een uitgebreide functie uitleg in Word, moet de code dankzij heldere naamgeving zelfverklarend zijn. Er wordt in codebase uitsluitend beknopte comments geplaatst om complex architecturale keuzes te verantwoorden. Voor de configuratie van de applicatie is een overzichtelijke README beschikbaar die nieuwe ontwikkelaars begeleidt bij het lokaal opzetten van de frontend, backend en database.

Documentatie voor de REST API:

De tool (Swagger) genereert automatisch een interactieve documentatie interface op basis van de code. Dit fungeert als het technische contract tussen backend en frontend, en maakt het voor ontwikkelaars en testers uiterst eenvoudig om de API endpoints direct te begrijpen en te valideren zonder externe documenten te hoeven raadplegen.

Gebruikersdocumentatie na oplevering:

Bij oplevering van het project wordt specifieke documentatie voorzien voor de eindgebruiker. Er wordt een heldere gebruikershandleiding opgeleverd in Word, waarin stapsgewijs wordt uitgelegd hoe acties zoals het aanmaken van instanties of het configureren van bugs in zijn werk gaan. Daarnaast zal deze documentatie ook direct raadpleegbaar zijn binnen de applicatie zelf.

Samenvatting

In opdracht van Brightest is de BrightSwagShop ontwikkeld, een uniek, fake e-commerce platform dat specifiek is ontworpen als trainingsomgeving voor testconsultants én als demonstratietool. Omdat bestaande demo applicaties vaak te stabiel zijn voor educatieve doeleinden, was er behoefte aan een platform waar defecten doelbewust en gecontroleerd geactiveerd kunnen worden. Hierdoor is het niet alleen de perfecte oefenomgeving, maar ook het ideale middel om tijdens demo's concreet te laten zien hoe softwaretesting in de praktijk werkt.

De kern van de oplossing is een fake webshop met een bijbehorend bug toggle dashboard voor trainers. Hiermee kunnen specifieke fouten in zowel de frontend als de backend worden ingeschakeld, die cursisten vervolgens moeten detecteren via manuele of geautomatiseerde testen. Om ervoor te zorgen dat verschillende trainingssessies onafhankelijk van elkaar kunnen draaien, is de applicatie gebouwd op een strikte multi-tenancy architectuur met volledige data isolatie.

De technische realisatie omvat een Next.js frontend, een Express backend en een serverless PostgreSQL database, waarbij het volledige uitroproces is geautomatiseerd via een CI/CD-pijplijn in GitHub Actions en Render.

Besluit

Met de voltooiing van de BrightSwagShop is een robuust en schaalbaar trainingsplatform opgeleverd dat direct inspeelt op de behoeften van Brightest. Het project toont aan dat het mogelijk is om een complexe fake e-commerce applicatie te bouwen die tegelijkertijd als veilige en controleerbare leeromgeving dient voor toekomstige testconsultants. De belangrijkste doelstellingen van deze bachelorproef zijn hierbij succesvol behaald. Er is een functioneel platform ontwikkeld met een uniek bug toggle dashboard, waardoor trainers fouten in de frontend en backend gericht kunnen activeren voor educatieve doeleinden. Verder garandeert de strikte multi-tenancy architectuur volledige data-isolatie via een unieke koppeling met de database instantie, waardoor meerdere trainingssessies gelijktijdig en onafhankelijk van elkaar kunnen plaatsvinden zonder risico op interferentie. Ook de volledige automatisering van het releaseproces via een CI/CD-pijplijn met GitHub Actions waarborgt de kwaliteit en stabiliteit van de omgeving door middel van geautomatiseerde testen.

Technisch gezien heeft het gebruik van een moderne stack bestaande uit Next.js, Express en Prisma gezorgd voor een performante en onderhoudbare applicatie. De keuze voor serverless hosting op platformen zoals Neon en Render maakt de oplossing bovendien kostenefficiënt en schaalbaar. Voor mij persoonlijk was deze stage bij Brightest een zeer leerrijke ervaring waarin ik de theoretische fundamenteën van mijn opleiding heb kunnen omzetten in een tastbaar product. Het project heeft mijn inzicht in zowel de architectuur van softwareontwikkeling als de cruciale rol van kwaliteitsbewaking enorm vergroot. Ik ben ervan overtuigd dat de BrightSwagShop een waardevol instrument zal blijven voor de verdere professionele groei van de consultants binnen Brightest.

Bronvermelding

- [1] Auth0. (2026). JSON Web Tokens. Auth0. <https://jwt.io/>
- [2] Docker. (2026). Docker Platform. Docker. <https://www.docker.com/>
- [3] GitHub. (2026). GitHub Actions. GitHub. <https://github.com/features/actions>
- [4] Google. (2026). Gemini [LLM, gebruikt voor zinsverbetering en spellingcontrole]. Google. <https://gemini.google.com/>
- [5] Google. (2026). Google Chrome. <https://www.google.com/chrome/>
- [6] Kelektiv. (2026). Bcrypt.js. Node.js. <https://www.npmjs.com/package/bcrypt>
- [7] Koyeb. (2026). Koyeb Serverless Platform. <https://www.koyeb.com/>
- [8] Neon. (2026). Neon Serverless Postgres. Neon. <https://neon.tech/>
- [9] OpenJS Foundation. (2026). Express. <https://expressjs.com/>
- [10] OpenJS Foundation. (2026). Jest. <https://jestjs.io/>
- [11] OpenJS Foundation. (2026). Node.js. <https://nodejs.org/>
- [12] Planttext. (2026). Planttext. <https://www.planttext.com>
- [13] Playwright. (2026). Playwright. <https://playwright.dev/>
- [14] Prisma. (2026). Prisma ORM. Prisma Data. <https://www.prisma.io/>
- [15] Render. (2026). Render Cloud Hosting. Render. <https://render.com/>
- [16] SmartBear Software. (2026). Swagger API Documentation / Swagger UI. <https://swagger.io/>
- [17] The PostgreSQL Global Development Group. (2026). PostgreSQL. <https://www.postgresql.org/>
- [18] Vercel. (2026). Next.js. Vercel. <https://nextjs.org/>
- [19] Vercel. (2026). Vercel Platform. Vercel. <https://vercel.com/>

Bijlagen

Omdat dit document als één gebundelde PDF wordt ingediend, zijn hieronder enkele visuele screenshots van de testrapportages opgenomen ter ondersteuning van het Testplan en Test Reflectieverslag. De originele, interactieve HTML-rapporten en de volledige codebase kunnen op verzoek worden gedemonstreerd tijdens de mondelinge verdediging.

End-To-End:

All 178

✓ Passed 178

Failed 0

Flaky 0

⌚ Skipped 21

🔄

⚙️

4/21/2026, 5:06:14 PM Total time: 14.6m

> auth.setup.ts

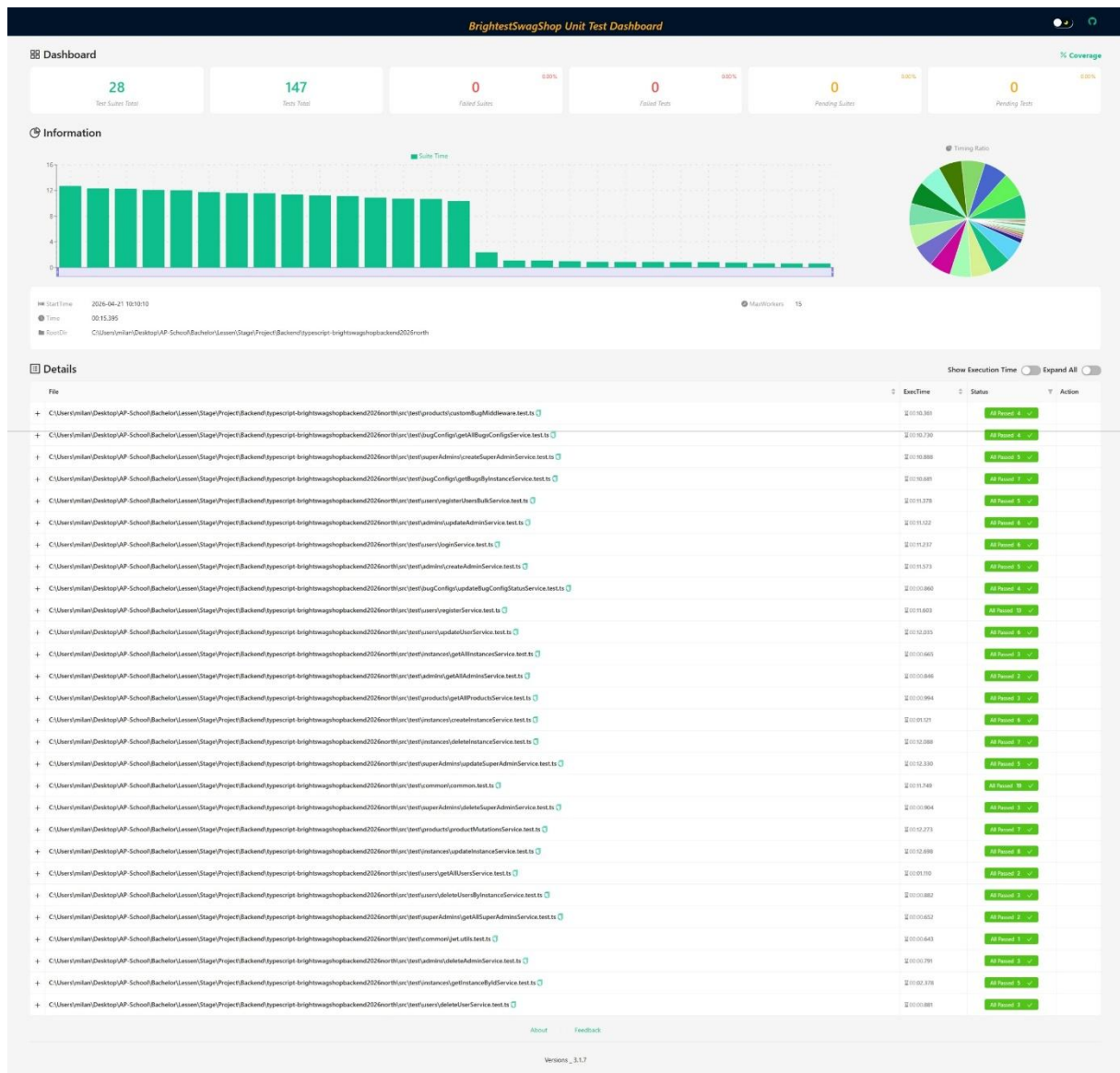
> instancePage.spec.ts

> login.spec.ts

> navbar.spec.ts

> productPage.spec.ts

Unit Testen:



All files

92.36% Statements [View](#) 82.48% Branches [View](#) 94.44% Functions [View](#) 94.89% Lines [View](#)

Press n or j to go to the next uncovered block, b or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
common	100%	55/55	100%	27/27
services/admins	54.72%	72/76	87.81%	38/66
services/admins/helpers	89.47%	17/19	80%	8/10
services/bug/configs	100%	38/39	100%	20/20
services/bug/configs/helpers	100%	4/4	100%	0/0
services/instances	87.5%	70/80	80.43%	37/46
services/instances/helpers	100%	4/4	100%	0/0
services/products	74.0%	47/63	50.98%	36/51
services/products/helpers	100%	5/5	100%	0/0
services/superAdmins	92.55%	56/71	86.2%	50/58
services/users	95.41%	125/131	87.85%	94/107
services/users/helpers	100%	16/16	100%	8/8

Code coverage generated by Istanbul at 2025-04-21 19:16:29.4712